同芯同行

Chip in, big win

# 2026年4月北美芯片设计/验证题目汇总

根据同学回忆整理 · 52道真题含答案与解析

Apple · Google · Nvidia · Qualcomm · Etched · D-Matrix

# 目 录


### SystemVerilog — Language Features

Q1. always_comb vs always @(*)

Q2. Array Reduction: Find element(s) with the smallest absolute value

Q3. Array Reduction: Find elements whose value equals their index

Q4. Array Reduction: Find duplicate elements

Q5. Array Reduction: sum() bit-width trap

Q6. Bitwise: Reverse nibble order 0xABCDEF -> 0xFEDCBA

Q7. Bitwise: Rotate left by 12 bits 0xABCDEF -> 0xDEFABC

Q8. Events: @event vs wait(event.triggered) and race conditions

### Algorithms

Q9. Verilog Coding: Rising Edge Detector

Q10. SV Coding: Fibonacci Sequence

### SystemVerilog — OOP

Q11. OOP: Constraint override and polymorphic randomize()

Q12. OOP: Handle vs Object (shallow copy trap)

Q13. Abstract class and pure virtual function

### Digital Design

Q14. SV OOP: Base/Child Class Polymorphism with virtual

### SystemVerilog — Constraints & Randomization

Q15. Randomization: Array size and element count constraints

Q16. Randomization: Uniqueness constraints (1D & 2D)

Q17. SV Constraint: Circular Right Shift Uniqueness

Q18. SV Constraint: Unique 2D Array (Row, Column, Diagonal) without unique keyword

Q19. SV: Generate a Random Number in a Range without $urandom_range()

Q20. SV Constraint: Weighted Distribution without dist keyword

### SystemVerilog — Assertions (SVA)

Q21. SVA: req/ack handshake protocol

Q22. SVA: Limit req rising edges before ack

Q24. SVA: Five-Requirement Assertion Using throughout and within

### SystemVerilog — Coding

Q25. SV Coding: Monitor with Mailbox on Valid-Ready Interface

### UVM

Q26. Virtual Sequencer vs Virtual Sequence

Q27. uvm_driver handshaking mechanism

Q28. UVM Factory: how it works

Q29. Running the same sequence concurrently on multiple agents

Q30. uvm_config_db: how it works

Q31. What happens inside UVM after run_test() is called

Q32. Write a simplified config_db class from scratch

Q33. UVM Coding: AHB Driver with Pipelined Address/Data Phases

Q34. UVM: Shallow Copy vs Deep Copy and Their Applications in Testbench

## Digital Design

Q35. Gray Code: Encode and decode
Q36. Clock Domain Crossing (CDC) techniques
Q37. Moore vs Mealy state machine
Q38. Timing issues and solutions
Q39. FSM design: output 1 when serial input (LSB first) is divisible by 5
Q40. FIFO depth calculation
Q41. Roofline Model analysis
Q42. Two types of power and how to optimize
Q43. Fixing setup/hold violations after tape-out
Q44. Computer Architecture: Pipeline Advantages and Hazards

## Verification

Q45. Verification: Async FIFO Gray Code Pointer Checker
Q46. Verification: How to Verify an Async FIFO

## Algorithms & Coding

Q47. Find the k-th smallest element in an array
Q48. Reverse a stack using recursion only
Q49. Maximum subarray sum (Kadane's Algorithm)

## UVM

Q50. OOP Design: Bicycle & Unicycle pricing system

## Brain Teasers

Q51. Brain teaser: measure 4L using 5L and 3L buckets
Q52. Brain Teaser: The Three Light Switches

## SystemVerilog — Language Features

### Q1. always_comb vs always @(*)

QUESTION

Explain the differences between always_comb and always @(*).

Cover: execution timing, sensitivity list inference, latch

inference behavior, behavior when calling tasks/functions,

and the difference between always_ff and always @(posedge clk).

ANSWER

```
Feature              | always_comb              | always @(*)
---------------------|--------------------------|--------------------------
Standard             | SystemVerilog only       | Verilog-2001
Executes at time 0   | Yes, automatically       | No, waits for first event
Sensitivity list     | Inferred at elaboration  | Inferred at runtime
Latch inference      | Tool errors/warns        | Silently infers latch
Task/function calls  | Includes internal signals| Only sees block-level sigs

always_ff enforces non-blocking assignments (<=) only;
always @(posedge clk) allows mixing = and <= silently,
which can cause race conditions.

Fix for always @(*) with tasks: use always_comb, which
automatically adds task-internal signals to sensitivity list.
```

解释

always_comb 是 SystemVerilog 的改进版，主要解决了三个 always@(*) 的问题：

1. 时间0执行：always_comb 在仿真开始时自动执行一次，always@(*) 不会，

这可能导致仿真开始时输出值为 X。

2. 敏感列表完整性：always_comb 会把 task 内部读取的信号也加入敏感列表，

always@(*) 只看 always 块本身可见的信号，调用 task 时容易漏掉信号。

3. Latch 检测：always_comb 要求所有分支都赋值，否则工具报错；

always@(*) 遇到 if 没有 else 时会静默推断出 latch。

面试重点：always_ff 强制使用 <=，工具会报错如果用了 =，这避免了

non-blocking/blocking 混用导致的 race condition。

### Q2. Array Reduction: Find element(s) with the smallest absolute value

QUESTION

Given the array below, find the element(s) with the smallest

absolute value using SystemVerilog array methods.

Explain whether .min() with clause returns the original element

or the transformed value.

int arr[] = '{3, -1, 4, -7, 2};

ANSWER

```
int arr[] = '{3, -1, 4, -7, 2};
int min_abs_queue[$];
int min_abs_val;
int min_abs_elements[$];

// .min() with (expr): uses expr as sort key,
// returns the ORIGINAL element whose key is smallest
// keys = {3,1,4,7,2} -> smallest key=1 -> original element=-1
min_abs_queue = arr.min() with (item < 0 ? -item : item);

// Recompute the reference absolute value
min_abs_val = min_abs_queue[0] < 0 ? -min_abs_queue[0]
                                   :  min_abs_queue[0];
// min_abs_val = 1

// Find ALL elements whose absolute value equals min_abs_val
min_abs_elements = arr.find() with (
    (item < 0 ? -item : item) == min_abs_val
);
// min_abs_elements = {-1}
```

解释

.min() with (expr) 这里有一个常见的陷阱：它用 expr 作为排序键，

但返回的是原始元素值，不是 expr 计算后的值。

比如 arr = {3, -1, 4, -7, 2}，绝对值最小的是 -1（绝对值为1），

.min() with (item < 0 ? -item : item) 返回的是 -1，而不是 1。

所以后面还要再算一次 min_abs_val 才能用来做 find() 的条件。

面试陷阱点就在这里：很多人以为 .min() with 返回的是变换后的值。

## Q3. Array Reduction: Find elements whose value equals their index

QUESTION

Given the array below, find all elements whose value equals

their index. Return both the matching values and their indices.

Use .find() and .find_index().

int arr[] = '{0, 5, 2, 7, 4};

ANSWER

```
int arr[] = '{0, 5, 2, 7, 4};
int matched_vals[$];
int matched_idxs[$];

// .find_index() returns indices where condition is true
matched_idxs = arr.find_index() with (item == item.index);

// .find() returns the element values themselves
matched_vals = arr.find() with (item == item.index);
```

```
// arr[0]=0, arr[2]=2, arr[4]=4
// matched_idxs = {0, 2, 4}
// matched_vals  = {0, 2, 4}
```

解释

item.index 是 SV 数组迭代器的特殊属性，表示当前元素的下标。

.find() 返回满足条件的元素值，.find_index() 返回满足条件的下标。

两者都返回 queue 类型。

注意：item.index 只在数组方法的 with 子句中有效，不能在外部使用。

## Q4. Array Reduction: Find duplicate elements

QUESTION

Given the array below, find all elements that appear more than

once. Provide at least two implementations.

int arr[] = '{1, 3, 1, 2, 3, 6};

ANSWER

```
// Method 1: Associative array count (most readable)
int count[int];
foreach (arr[i]) count[arr[i]]++;
foreach (count[k])
  if (count[k] > 1) $display("Duplicate: %0d", k);

// Method 2: Sort + adjacent comparison
int sorted[] = arr;
sorted.sort();
foreach (sorted[i])
  if (i > 0 && sorted[i] == sorted[i-1])
    duplicates.push_back(sorted[i]);

// Method 3: unique() + find_index()
int unique_arr[$] = arr.unique();
foreach (unique_arr[i]) begin
  int val = unique_arr[i];
  int idxs[$] = arr.find_index() with (item == val);
  if (idxs.size() > 1) duplicates.push_back(val);
end
// Result: duplicates = {1, 3}
```

解释

三种方法各有优劣：

方法1（关联数组）：最直观，O(n)，推荐首选。

方法2（排序+相邻比较）：O(n log n)，但注意 .sort() 会修改原数组，

需要先复制一份。

方法3（unique + find_index）：最符合 SV 风格，但复杂度较高。

面试中方法1最容易写出来，建议优先掌握。

## Q5. Array Reduction: sum() bit-width trap

QUESTION

What is wrong with the following code? Explain the root cause

and provide the fix.

bit [3:0] arr[] = '{4'd10, 4'd12, 4'd15};

int total;

total = arr.sum();

ANSWER

```
Problem: .sum() accumulates using the element's own bit width
(4 bits here). 10+12+15 = 37, but 4-bit max is 15.
Truncation: 37 % 16 = 5. Assigning to int AFTER the fact
does not recover the lost bits -- the damage is already done
during accumulation.

Fix:
total = arr.sum() with (int'(item));
// Promotes each element to int BEFORE accumulation -> result = 37
```

解释

这是一个很经典的陷阱：.sum() 的累加是用元素自身的位宽来做的，

不是用接收结果的变量位宽。4-bit 的元素累加时最大只能是15，

所以 10+12+15=37 在 4-bit 里就变成了 37%16=5。

即使最后赋值给 int，溢出已经发生，数据已经丢失。

正确做法是在 with 子句里用 int'(item) 先做类型提升，

这样每次累加都在 32-bit int 的空间里进行，不会溢出。

## Q6. Bitwise: Reverse nibble order 0xABCDEF -> 0xFEDCBA

QUESTION

Reverse the nibble (4-bit) order of a 24-bit value 0xABCDEF

to get 0xFEDCBA. Provide at least two implementations.

ANSWER

```
logic [23:0] in  = 24'hABCDEF;
logic [23:0] out;

// Method 1: Explicit bit concatenation
out = { in[3:0], in[7:4], in[11:8],
        in[15:12], in[19:16], in[23:20] };
// out = 0xFEDCBA

// Method 2: Streaming operator (cleaner, scales automatically)
out = {<<4{in}};
// {<<4{in}} reverses in 4-bit chunks -> 0xFEDCBA

// Bonus: reverse every individual bit
out = {<<{in}};
// reverses bit by bit -> 0xF7B3D5
```

解释

方法1是最直白的写法，手动列出每个 nibble 的位置并重新拼接。

方法2使用 SV 的 streaming operator {<<N{expr}}，

<<N 表示以 N bit 为单位做翻转，非常简洁，面试中写出这个会加分。

注意区分 {<<4{in}}（nibble 级翻转）和 {<<{in}}（bit 级翻转）的结果不同。

## Q7. Bitwise: Rotate left by 12 bits 0xABCDEF -> 0xDEFABC

QUESTION

Rotate a 24-bit value 0xABCDEF left by 12 bits to get 0xDEFABC.

Provide at least two implementations.

ANSWER

```
logic [23:0] in  = 24'hABCDEF;
logic [23:0] out;

// Method 1: Part-select concatenation
out = { in[11:0], in[23:12] };
// { DEF, ABC } = 0xDEFABC

// Method 2: Streaming operator
out = {<<12{in}};
// reverses in 12-bit chunks: {ABC, DEF} -> {DEF, ABC}

// Method 3: Shift operations
out = (in << 12) | (in >> 12);
```

解释

循环移位本质上是把高位和低位对调，所以最直观的是方法1：

把低12位拼接到高12位前面。

方法2用 streaming operator {<<12{in}} 以12-bit为单位翻转，

对于24-bit数就是交换前后两半，效果等同于循环左移12位。

方法3用移位+或运算，是 C/C++ 程序员最熟悉的写法，

在 SV 中同样有效，注意 >> 是逻辑右移，高位补0。

## Q8. Events: @event vs wait(event.triggered) and race conditions

QUESTION

(a) What is the difference between @event and wait(event.triggered)?

In what scenario does each cause problems?

(b) How does ->> (nonblocking trigger) solve the race condition?

(c) Compare race-safety of these four combinations:

(1) -> + @e (2) ->> + @e (3) -> + wait(e.triggered) (4) ->> + wait(e.triggered)

ANSWER

```
// @e is EDGE-SENSITIVE: blocks until the NEXT trigger of e.
// If -> e fires in the same time step before @e executes, it MISSES it.
```

```
// wait(e.triggered) is LEVEL-SENSITIVE to the current time step:
// If -> e already fired in this step, wait() returns IMMEDIATELY.

// Race example:
event e;
fork
  begin #10; -> e;  end              // triggers at t=10 (Active region)
  begin #10; @e; $display("@e"); end // MAY MISS (same delta)
  begin #10; wait(e.triggered);
             $display("wait"); end  // will NOT miss
join

// ->> fires in NBA region (after Active region settles).
// @e threads in Active region have already registered by then -> safe.

// Race-safety summary:
// Trigger | Wait construct      | Race-safe?
// --------|---------------------|------------
// ->      | @e                  | NO  (same delta can miss)
// ->>     | @e                  | YES (NBA fires after Active)
// ->      | wait(e.triggered)   | YES (level-sensitive)
// ->>     | wait(e.triggered)   | YES (both safe)
```

解释

SV 的仿真调度分多个 region：Active -> NBA -> Observed -> Reactive...

-> 在 Active region 触发 event，如果同一时间步里 @e 还没来得及注册，

就会错过这次触发，这就是竞争条件。

wait(e.triggered) 是电平敏感的：只要在当前时间步内 event 曾经被触发，

triggered 属性就是 true，wait 立即返回，不会错过。

->> 把触发推迟到 NBA region，此时 Active region 的所有 @e 线程

已经注册完毕，所以不会错过。

面试建议：用 -> 触发时配合 wait(e.triggered)；

用 @e 等待时配合 ->> 触发，两种组合都是 race-safe 的。

## Algorithms

### Q9. Verilog Coding: Rising Edge Detector

QUESTION

Design a rising edge detector circuit in Verilog/SystemVerilog.

The circuit should:

- Take an asynchronous input signal 'din' and a clock 'clk'

- Output a single-cycle pulse 'rising_edge' that is HIGH for

exactly one clock cycle whenever a rising edge on 'din' is detected

- Be synchronous to the clock (output is registered)

Draw the timing diagram and explain the underlying principle.

Timing diagram:

clk : _/_/_/_/_/_/_/_/ din : ____/^^^^^^^^^_________

din_d1 : _____/^^^^^^^^^_______

rising_edge: _____/_________________

ANSWER

```
// Method 1: Single flip-flop delay + combinational detection
module rising_edge_detector (
  input  logic clk,
  input  logic rst_n,
  input  logic din,
  output logic rising_edge
);
  logic din_d1;  // din delayed by one clock cycle

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) din_d1 <= 1'b0;
    else        din_d1 <= din;
  end

  // Rising edge: current cycle is HIGH, previous cycle was LOW
  assign rising_edge = din & ~din_d1;

endmodule

// Method 2: Fully registered output (pulse is one cycle later)
module rising_edge_detector_reg (
  input  logic clk,
  input  logic rst_n,
  input  logic din,
  output logic rising_edge
);
  logic din_d1;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      din_d1      <= 1'b0;
      rising_edge <= 1'b0;
    end else begin
      din_d1      <= din;
      rising_edge <= din & ~din_d1;  // registered: glitch-free output
    end
  end

endmodule

// Falling edge detector (for reference):
// assign falling_edge = ~din & din_d1;

// Both edge detector:
// assign any_edge = din ^ din_d1;
```

解释

边沿检测的核心原理：用一个寄存器把信号打一拍，然后对比当前值和延迟一拍的值。

上升沿的条件：当前拍 din=1，上一拍 din_d1=0，即 din & ~din_d1。

输出是宽度为一个时钒周期的脉冲。

方法一（组合逻辑输出）和方法二（寄存器输出）的区别：

方法一： rising_edge 由 assign 驱动，属于组合逻辑输出。

优点：延迟更小（同一拍就能输出）。

缺点：如果 din 有毛刺，输出也会有毛刺。

方法二： rising_edge 由 always_ff 驱动，属于寄存器输出。

优点：输出完全同步，无毛刺，时序更容易分析。

缺点：相对方法一多一个周期的延迟。

下降沿检测：~din & din_d1（当前为0，上一拍为1）。

双沿检测：din ^ din_d1（任意变化就输出脉冲）。

面试重点：能快速写出 always_ff + assign
的组合，并解释为什么不能用纯组合逻辑实现（没有寄存器打拍，输出不是一个封阅的周期脉冲）。

## Q10. SV Coding: Fibonacci Sequence

QUESTION

Implement the Fibonacci sequence in SystemVerilog.

Output the first N Fibonacci numbers.

Provide both iterative and recursive implementations.

ANSWER

```
// Method 1: Iterative (preferred -- no stack overflow risk)
function automatic void fib_iterative(int n);
  int a = 0, b = 1, tmp;
  $display("Fib[0] = %0d", a);
  if (n == 0) return;
  $display("Fib[1] = %0d", b);
  for (int i = 2; i <= n; i++) begin
    tmp = a + b;
    a   = b;
    b   = tmp;
    $display("Fib[%0d] = %0d", i, b);
  end
endfunction

// Method 2: Recursive (clean but O(2^n) without memoization)
function automatic int fib_recursive(int n);
  if (n <= 0) return 0;
  if (n == 1) return 1;
  return fib_recursive(n-1) + fib_recursive(n-2);
endfunction

// Usage in testbench:
initial begin
  fib_iterative(10);
  for (int i = 0; i <= 10; i++)
    $display("fib(%0d) = %0d", i, fib_recursive(i));
end

// Output: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55
```

解释

迭代法是首选：O(n) 时间和 O(1) 空间，无递归层栈溢出风险。

递归法直观但效率低：O(2^n) 时间，因为大量重复计算。

面试建议：先写迭代，再说明能用递归但复杂度更高，展示对两种方法的理解。

关键字 automatic 必不可少：如果这两个函数写在 module 里，必须加 automatic，否则递归会出错（static 存储共享变量）。

## SystemVerilog — OOP

### Q11. OOP: Constraint override and polymorphic randomize()

QUESTION

What are the possible ranges of x and y after execution?

Analyze: (1) same-name constraint override rule,

(2) legality of upcast, (3) virtual method dispatch of randomize(),

(4) what happens when accessing y through a Base handle.

class Base;

```
rand bit [7:0] x;
constraint c { x <= 50; }
```

endclass

class Child extends Base;

```
rand bit [7:0] y;
constraint c  { x > 100; }  // same name as Base::c
constraint c2 { y == x + 1; }
```

endclass

module tb;

Base b; Child c;

initial begin

c = new();

b = c; // upcast: always legal, no $cast needed

void'(b.randomize());

```
end
```

endmodule

ANSWER

```
(1) Constraint override: Child's 'c' replaces Base's 'c' entirely.
    Only Child constraint c (x > 100) is active. Base's x <= 50
    is gone. Both cannot coexist under the same name.

(2) Upcast (b = c): always legal in SV OOP, no $cast required.
    b now points to the Child object.

(3) randomize() is virtual: even through Base handle b, it calls
```

```
    Child's randomize. All Child constraints are active:
      x : 101 ~ 255
      y : x + 1 (with overflow: if x=255, y=0)

(4) b.y -> COMPILE ERROR: Base handle has no 'y' field.
    Must downcast first: Child tmp; $cast(tmp, b); tmp.y;
```

解释

这道题考查了 OOP 的三个核心概念：

1. 同名约束覆盖：子类和父类定义了同名的 constraint，子类的会完全覆盖父类的，父类的约束就消失了，不是两个同时生效。

2. Upcast（向上转型）：把 Child 句柄赋给 Base 句柄永远合法，不需要 $cast。反过来（downcast）才需要 $cast，且可能失败。

3. randomize() 是虚方法：即使通过 Base 句柄调用，运行时会找到实际对象（Child）的 randomize 实现，所以 Child 的所有约束都生效。

面试重点：通过 Base 句柄访问 y 会编译报错，要用 $cast 做向下转型才能访问。

## Q12. OOP: Handle vs Object (shallow copy trap)

QUESTION

What is the relationship between p2 and p1 in the code below?

What happens after randomize() is called?

Further explain the difference between:

(a) p2 = p1 (b) p2 = new p1 (c) passing handle via ref parameter

Packet p1 = new();

Packet p2 = p1;

void'(p2.randomize());

ANSWER

```
p2 = p1 is a HANDLE COPY, not an object copy.
Both p1 and p2 point to the SAME object on the heap.
Calling p2.randomize() also randomizes p1's fields.

(a) p2 = p1       -> handle copy; same object; p1 fields also change
(b) p2 = new p1   -> copy constructor; NEW object with p1's current
                     field values; p1 and p2 are independent after
(c) ref parameter -> the function can modify the caller's handle itself
    function void bad (Packet h);   // local copy of handle
      h = new();  // only changes local h, caller unchanged
    endfunction
    function void good(ref Packet h);  // modifies caller's handle
      h = new();  // caller now sees the new object
    endfunction
```

解释

SV 中的 class 变量本质上是一个指针/句柄，不是对象本身。

p2 = p1 只是复制了这个指针，两个变量指向堆内存中同一个对象。

所以对 p2 做任何操作（randomize、修改字段）都会影响 p1。

p2 = new p1 才是真正的浅拷贝：在堆上新建一个对象，

把 p1 所有字段的值复制过去，之后两者互不影响。

注意：如果字段本身是 class 句柄（嵌套对象），new p1 也只复制句柄，

需要深拷贝才能完全独立。

## Q13. Abstract class and pure virtual function

QUESTION

Answer the following about abstract classes in SystemVerilog:

(a) What is a virtual class? How does it differ from a normal class?

Why can it not be instantiated directly?

(b) What is a pure virtual function? What happens if a subclass

does not implement all pure virtual functions?

(c) Which UVM classes use abstract class design? Give examples.

(d) Write abstract class base_driver with pure virtual task

drive_item(), and derive axi_driver and apb_driver.

ANSWER

```
// (a) virtual class = abstract class
//   Cannot be instantiated with new() directly.
//   Exists only to define an interface/contract for subclasses.
//   Normal class: can be instantiated; methods have implementations.
//   virtual class: may have pure virtual methods with no body.

// (b) pure virtual function:
//   Declared with 'pure virtual', no implementation in base class.
//   Any class with a pure virtual method becomes abstract.
//   A subclass that does not implement ALL pure virtuals also
//   becomes abstract and cannot be instantiated.

// (c) UVM examples of abstract class design:
//   uvm_object      : base for all UVM objects
//   uvm_component   : base for all UVM components
//   uvm_sequence_base: defines sequence contract
//   uvm_driver      : defines run_phase and seq_item_port interface

// (d) Implementation:
virtual class base_driver;
  pure virtual task drive_item(input my_item item);
endclass

class axi_driver extends base_driver;
  virtual task drive_item(input my_item item);
    @(posedge vif.clk);
    vif.awvalid <= 1;
    vif.awaddr  <= item.addr;
    vif.wdata   <= item.data;
  endtask
endclass

class apb_driver extends base_driver;
  virtual task drive_item(input my_item item);
```

```
    @(posedge vif.pclk);
    vif.psel   <= 1;
    vif.paddr  <= item.addr;
    vif.pwdata <= item.data;
  endtask
endclass
```

解释

virtual class 是 SV 对 OOP 中抽象类的实现。

它的作用是定义"接口契约"：声明子类必须实现的方法，但不提供实现。

pure virtual 方法没有函数体，子类必须全部实现，否则子类也变成抽象类，

也不能实例化，工具会在编译时报错。

UVM 框架本身大量使用了这种设计，比如 uvm_object 定义了

copy, compare, print 等虚方法，让所有子类都有统一的接口。

面试中常考的还有 virtual class 和 interface class 的区别：

interface class（SV 2012引入）只能包含 pure virtual 方法，

更接近 Java 的 interface，而 virtual class 可以有具体实现。

## Digital Design

### Q14. SV OOP: Base/Child Class Polymorphism with virtual

QUESTION

Given the following class hierarchy, answer the three follow-up

questions about static vs virtual dispatch:

class base_class;

```
int address;
int data;
function new();
  address = 32'hA000; data = 8'hFF;
endfunction
function void display();
  $display("Base - address: %0h, data: %0h", address, data);
endfunction
```

endclass

class child_class extends base_class;

```
function new();
  super.new();
  address = 32'hB000; data = 8'h55;
endfunction
function void display();
  $display("Child - address: %0h, data: %0h", address, data);
endfunction
```

endclass

```
module tb;
initial begin
base_class b = new();
child_class c = new();
b.display(); // Q1
c.display(); // Q1
b = c;
b.display(); // Q3
end
endmodule
```

Q1: What does each display() call print? Which function is called?

Q2: What changes if display() is declared virtual in base_class?

Q3: After b = c, what does b.display() print?

What changes with virtual?

ANSWER

```
// Q1: Without virtual (static dispatch, resolved at COMPILE TIME)
//
//   b.display() -> calls base_class::display()
//   Prints: "Base - address: a000, data: ff"
//
//   c.display() -> calls child_class::display()
//   Prints: "Child - address: b000, data: 55"
//
//   Static dispatch: the function called depends on the HANDLE TYPE,
//   not the object type. b is declared as base_class -> base version.

// Q2: With "virtual function void display()" in base_class
//
//   b.display() -> still prints "Base - address: a000, data: ff"
//   c.display() -> still prints "Child - address: b000, data: 55"
//   No difference for Q1, because b points to a base_class object
//   and c points to a child_class object. Virtual only matters when
//   the handle type differs from the actual object type (see Q3).

// Q3: After b = c  (upcast: base_class handle points to child object)
//
//   WITHOUT virtual:
//     b.display() -> static dispatch on handle type (base_class)
//     Calls base_class::display()
//     Prints: "Base - address: b000, data: 55"
//     NOTE: prints Base label but child's VALUES (b000, 55),
//     because the object IS a child_class with child field values.
//
//   WITH virtual:
//     b.display() -> dynamic dispatch on ACTUAL object type (child)
//     Calls child_class::display()
//     Prints: "Child - address: b000, data: 55"

// Summary table:
// Handle | Object | virtual? | Function called        | Output
// -------|--------|----------|------------------------|------------------
// base   | base   | no       | base_class::display()  | Base a000 ff
```

```
// child  | child  | no       | child_class::display() | Child b000 55
// base   | child  | no       | base_class::display()  | Base b000 55  <-KEY
// base   | child  | yes      | child_class::display() | Child b000 55 <-KEY
```

解释

这道题考查 SV OOP 中静态绑定和动态绑定的核心区别。

不加 virtual：静态绑定，编译时决定调用哪个函数。

决定因素是句柄的声明类型（即 base_class b），与实际对象类型无关。

b = c 后 b.display() 用的是 Base 的 display 函数，

但打印的是 child 的字段值（b000, 55），因为字段访问的是实际对象。

这个细节尤其容易踩陷，面试的考点所在。

加了 virtual：动态绑定，运行时根据实际对象类型决定。

b = c 后 b 指向的是 child 对象，所以 b.display() 调用 child 的版本。

这就是多态的本质：同一个调用形式，根据对象实际类型表现不同。

UVM 中大量使用 virtual function（如 do_print, do_copy, do_compare）就是这个原理。

## SystemVerilog — Constraints & Randomization

### Q15. Randomization: Array size and element count constraints

QUESTION

Write randomization constraints to satisfy all of the following:

- rand int arr[] has size between 4 and 8

- Exactly 3 elements equal 100

- Explain how arr.sum() with (int'(item == 100)) works as a counter

ANSWER

```
class MyClass;
  rand int arr[];
  constraint c_size  { arr.size() inside {[4:8]}; }
  constraint c_count { arr.sum() with (int'(item == 100)) == 3; }
endclass

// How the counter works:
// (item == 100) evaluates to 1 (true) or 0 (false) per element
// int'(...) promotes to int to avoid bit-width truncation
// .sum() with (...) adds up all per-element results
// == 3 means exactly 3 elements satisfy the condition
```

解释

arr.sum() with (int'(item == 100)) 是一个非常经典的 SV 约束技巧。

原理：(item == 100) 对每个元素返回 1 或 0，

sum() 把所有的 1 和 0 加起来，就等于满足条件的元素个数。

必须用 int'() 做类型提升，否则 bit 类型的累加会溢出（同 Q5 的问题）。

面试中这个写法经常考，要熟练掌握。

## Q16. Randomization: Uniqueness constraints (1D & 2D)

QUESTION

Write constraints to ensure:

(a) All elements in a 1D array are unique (use unique keyword)

(b) All elements across a 2D array int arr[3][4] are unique

(use foreach nesting)

Explain the semantic difference of unique in 1D vs 2D.

ANSWER

```
// (a) 1D unique
class My1D;
  rand int arr[];
  constraint c_size   { arr.size() == 6; }
  constraint c_unique { unique {arr}; }
endclass

// (b) 2D unique across all elements
class My2D;
  rand int arr[3][4];
  constraint c_unique {
    foreach (arr[i,j])
      foreach (arr[k,l])
        if (!(i==k && j==l)) arr[i][j] != arr[k][l];
  }
endclass

// Semantic difference:
// unique {arr}  -> works on 1D queues/dynamic arrays directly
// For 2D, unique applies per-row only if written as unique {arr[i]}
// To enforce global uniqueness across all rows, must use foreach nesting
```

解释

unique 关键字在 SV 约束中只对一维数组有效，

写 unique {arr} 对2D数组时，语义上只保证同一行内唯一，

无法保证跨行唯一。

要保证2D数组所有元素全局唯一，必须用 foreach 双重嵌套，

对每一对 (i,j) 和 (k,l) 约束 arr[i][j] != arr[k][l]。

这个约束的约束求解开销很大，实际项目中要谨慎使用。

## Q17. SV Constraint: Circular Right Shift Uniqueness

QUESTION

Randomly generate an 8-bit number such that all values in its

circular right shift sequence are unique (no repeated value).

Example of an INVALID number:

8'hAA -> rotate right -> 8'h55 -> rotate right -> 8'hAA (repeated!)

Example of a VALID number:

8'hB4 -> 8'h5A -> 8'h2D -> ... -> all 8 rotations are distinct

ANSWER

```
// A circular right shift of an N-bit value by k bits is unique
// for all k in [0, N-1] if and only if the value has no rotational
// symmetry -- i.e. no shorter repeating pattern divides 8 bits.
// Values like 8'hAA (10101010) repeat after 2 shifts.
// Values like 8'hFF (11111111) repeat after 1 shift.
// We use a constraint that checks all 8 rotations are distinct.

class circ_shift_unique;
  rand bit [7:0] val;

  // Helper function: circular right shift by k bits
  function bit [7:0] rotr(bit [7:0] v, int k);
    return (v >> k) | (v << (8 - k));
  endfunction

  constraint c_unique_rotations {
    // For every pair (i, j) where i != j, the i-th and j-th
    // right-rotations of val must be different.
    foreach (val[i])   // iterate i from 0 to 7
      foreach (val[j]) // iterate j from 0 to 7
        if (i != j)
          rotr(val, i) != rotr(val, j);
  }
endclass

// Simpler post-randomize check approach (if constraints are too slow):
class circ_shift_unique_v2;
  rand bit [7:0] val;

  function void post_randomize();
    bit [7:0] seen[8];
    for (int k = 0; k < 8; k++) begin
      bit [7:0] rotated = (val >> k) | (val << (8-k));
      foreach (seen[j])
        if (j < k && seen[j] == rotated) begin
          // Collision found -- reject and re-randomize
          if (!this.randomize()) `uvm_fatal("RAND","")
          return;
        end
      seen[k] = rotated;
    end
  endfunction
endclass
```

解释

这道题的本质是找到一个没有旋转对称性的 8-bit 数。

8'hAA = 10101010 就是有对称性的例子：它周期为2，8次循环只有 2 种不同的值。

方法一：用 constraint 直接对所有 8 个旋转结果两两约束不相等。

实现上这个 constraint 的求解开销可能较大，因为需要展开 8x8 的对比。

方法二： post_randomize 里验证所有旋转是否唯一，不满足则重新随机化，更简单可任。

面试重点：能识别循环移位的对称性概念，并说明两种实现方式的权衡。

## Q18. SV Constraint: Unique 2D Array (Row, Column, Diagonal) without unique keyword

QUESTION

Use constraints to randomize a 4x4 2D array such that all values

are unique within each row, each column, and each diagonal.

You cannot use the 'unique' keyword.

ANSWER

```
class unique_2d;
  rand int arr[4][4];

  // Each value fits in range [1:16] (optional but cleaner)
  constraint c_range {
    foreach (arr[i,j]) arr[i][j] inside {[1:16]};
  }

  // Row uniqueness: no two elements in the same row are equal
  constraint c_row {
    foreach (arr[i,j])
      foreach (arr[i,k])
        if (j != k) arr[i][j] != arr[i][k];
  }

  // Column uniqueness: no two elements in the same column are equal
  constraint c_col {
    foreach (arr[i,j])
      foreach (arr[k,j])
        if (i != k) arr[i][j] != arr[k][j];
  }

  // Main diagonal (top-left to bottom-right): arr[0][0], [1][1]...
  constraint c_diag_main {
    foreach (arr[i,j])
      foreach (arr[k,l])
        if (i == j && k == l && i != k) arr[i][j] != arr[k][l];
  }

  // Anti-diagonal (top-right to bottom-left): arr[0][3], [1][2]...
  // Elements on anti-diagonal satisfy: i + j == N-1 (here N=4, so i+j==3)
  constraint c_diag_anti {
    foreach (arr[i,j])
      foreach (arr[k,l])
        if ((i+j == 3) && (k+l == 3) && !(i==k && j==l))
          arr[i][j] != arr[k][l];
  }

endclass
```

解释

这道题类似数独（Sudoku）的约束问题。

不能用 unique 关键字，就需要用双层 foreach 手动展开所有不等关系。

主对角线：i == j 的元素（arr[0][0], arr[1][1], arr[2][2], arr[3][3]）。

副对角线：i + j == N-1 的元素（arr[0][3], arr[1][2], arr[2][1], arr[3][0]）。

注意：这种约束的求解开销极大，实际项目中建议用 solve/before 指导求解顺序或结合 post_randomize 使用回溯算法。

## Q19. SV: Generate a Random Number in a Range without $urandom_range()

QUESTION

How do you generate a random integer within a specific range

[lo, hi] without using $urandom_range()?

Provide multiple methods.

ANSWER

```
// Method 1: $urandom + modulo  (most common interview answer)
// Generates value in [0, hi-lo], then shift by lo
int lo = 10, hi = 50;
int result;
result = (int'($urandom()) % (hi - lo + 1)) + lo;
// Range: [lo, hi] inclusive
// Caveat: slight modulo bias if (hi-lo+1) does not divide 2^32 evenly

// Method 2: $random + modulo  (older Verilog style)
result = ($random() % (hi - lo + 1)) + lo;
// $random returns a signed 32-bit value; use $abs or mask if needed

// Method 3: std::randomize() with constraint (cleanest SV style)
int val;
if (!std::randomize(val) with { val inside {[lo:hi]}; })
  $fatal("randomize failed");

// Method 4: rand variable with constraint
class range_rand;
  rand int val;
  int lo, hi;
  constraint c_range { val inside {[lo:hi]}; }
endclass

// Method 5: bit masking (when range is power of 2)
// For range [0, 2^N - 1]: mask with N-bit mask
result = $urandom() & 8'hFF;  // [0, 255]
```

解释

方法一和二是最快写出的，但有轻微的模数偏差问题：

如果范围大小不能整除 2^32，某些值的出现概率会稍高，面试时详细的考官可能会问到这个细节。

方法三（std::randomize with constraint）是最符合 SV 风格的写法，没有偏差问题。

面试建议：先说方法一（完成题目要求），再说明它的偏差问题，再给出 std::randomize 作为更好的替代方案。

## Q20. SV Constraint: Weighted Distribution without dist keyword

QUESTION

How do you implement a weighted distribution constraint without

using the 'dist' keyword?

Example target: value=0 with 50% probability,

value=1 with 30% probability,

value=2 with 20% probability.

ANSWER

```
// Method 1: Intermediate rand variable as "bucket"
// Use a rand selector in [0,99] and map ranges to values.
class weighted_dist;
  rand int unsigned selector;  // uniform in [0,99]
  rand int value;

  constraint c_selector { selector inside {[0:99]}; }

  constraint c_value {
    // Map selector ranges to values:
    // [0,49]  -> value=0  (50%)
    // [50,79] -> value=1  (30%)
    // [80,99] -> value=2  (20%)
    (selector inside {[0:49]})  -> (value == 0);
    (selector inside {[50:79]}) -> (value == 1);
    (selector inside {[80:99]}) -> (value == 2);
  }
endclass

// Method 2: solve...before to control distribution order
class weighted_dist_v2;
  rand bit [1:0] bucket;  // 4 equal buckets: 0,1,2,3
  rand int       value;

  constraint c_solve { solve bucket before value; }
  constraint c_map {
    // bucket 0,1 -> value 0 (50%)
    // bucket 2   -> value 1 (25%) -- approximate
    // bucket 3   -> value 2 (25%) -- approximate
    (bucket == 2'b00 || bucket == 2'b01) -> value == 0;
    (bucket == 2'b10)                    -> value == 1;
    (bucket == 2'b11)                    -> value == 2;
  }
endclass

// For reference: equivalent using dist keyword
// constraint c_dist { value dist {0:=50, 1:=30, 2:=20}; }
```

解释

方法一是最精确也最常考的写法：

先用一个均匀随机的 selector 变量将 [0,99] 分成几段，

每段映射到一个具体的 value，每段的长度就是对应的概率。

这种写法本质上是将分布问题转化为区间映射问题，通用且直观。

solve...before 强制求解器先确定 bucket，再根据 bucket 确定 value，

避免求解器自己调整分布导致概率偏差。

## SystemVerilog — Assertions (SVA)

### Q21. SVA: req/ack handshake protocol

QUESTION

Write an SVA assertion for the following handshake protocol:

- After req rises, ack must arrive within 2 to 5 cycles

- req must stay low while waiting for ack

Explain why |=> is used instead of |->, and the role of throughout.

ANSWER

```
property req_ack_protocol;
  @(posedge clk) disable iff (!rst_n)
  // |=> skips the cycle where req is high (just rose)
  // then checks req stays low for 1~4 more cycles until ack arrives
  // total delay from $rose(req) to $rose(ack) = 2~5 cycles
  $rose(req) |=> ((!req) throughout ##[1:4] $rose(ack)) ##1 $fell(ack);
endproperty
assert property (req_ack_protocol)
  else $error("req_ack_protocol FAILED");

// Cycle-by-cycle breakdown:
// Cycle 0: $rose(req) detected  [antecedent matches]
// Cycle 1: |=> jumps here; start of consequent
//          (!req) must be true (req is low)
//          throughout begins
// Cycle 2~4: (!req) must remain true throughout
// Cycle 2~5: $rose(ack) must occur somewhere in this window
//            (##[1:4] relative to cycle 1 = cycles 2~5 overall)
// Cycle after $rose(ack): $fell(ack) must occur
//   (ack was a single-cycle pulse, must go low next cycle)

// Why |=> instead of |->?
// At cycle 0, $rose(req) fires but req is still HIGH at that moment.
// |-> would immediately check (!req) in cycle 0 -> false -> FAIL.
// |=> advances one cycle, so throughout starts at cycle 1 where
// req should already be low.

// Why ##1 $fell(ack) at the end?
// Verifies ack is a single-cycle pulse: it must go low the cycle
// immediately after it rose. Without this, a stuck-high ack passes.

// throughout semantics:
// (!req) throughout ##[1:4] $rose(ack)
// means: (!req) must be true on EVERY cycle from the start of the
// sequence until (and including) the cycle $rose(ack) is matched.
```

解释

这道题的完整版 property 比简化版多了两个细节，都很重要：

1. |=> 而非 |-> 的原因：

$rose(req) 匹配的那个周期，req 本身还是高电平（刚从低变高）。

如果用 |->, consequent 在同一周期开始检查 !req，立即为 false，

assertion 永远失败。用 |=> 跳过这一拍，从下一拍（req 已变低）开始检查。

2. 时序计算：

Cycle 0: $rose(req) 触发

Cycle 1: |=> 跳到这里，throughout 开始，req 必须为低

Cycle 2~5: $rose(ack) 必须在这个窗口内出现

（##[1:4] 相对于 cycle 1，绝对位置是 cycle 2~5）

因此 $rose(req) 到 $rose(ack) 的总延迟 = 2~5 个周期，符合题意。

3. ##1 $fell(ack) 的作用：

验证 ack 是单周期脉冲，$rose(ack) 的下一拍必须 $fell(ack)。

没有这一项的话，ack 一直拉高也能通过 assertion，逻辑不完整。

4. throughout 的语义：

(!req) throughout ##[1:4] $rose(ack) 表示

在整个 ##[1:4] 序列匹配过程中的每一个周期，(!req) 都必须为真，

相当于一个贯穿整个等待窗口的持续监控条件。

## Q22. SVA: Limit req rising edges before ack

QUESTION

Write an SVA assertion: before ack arrives, req must not rise

more than 3 times (including the first, so <= 4 total).

Explain why the counter resets using $past(ack) instead of ack.

ANSWER

```
int req_cnt;
always @(posedge clk) begin
  if (!rst_n)          req_cnt <= 0;
  else if ($past(ack)) req_cnt <= 0;  // reset ONE cycle after ack
  else if ($rose(req)) req_cnt <= req_cnt + 1;
end

property p_max_req_count;
  @(posedge clk) disable iff (!rst_n)
  $rose(ack) |-> (req_cnt <= 4);
endproperty
assert property (p_max_req_count)
  else $error("FAILED: req_cnt = %0d", req_cnt);

// Why $past(ack) not ack? -- detailed analysis
//
// === Sampling analysis (does the choice affect the assertion?) ===
//
// SVA samples signals in the Preponed region:
//   Preponed -> Active -> NBA -> Observed -> Reactive
//   |                    |         |
//   SVA samples        always      SVA evaluates
//   req_cnt here       updates     boolean here
//                      req_cnt
//
// At the posedge clk where $rose(ack) triggers:
//   - SVA samples req_cnt in PREPONED (the old, pre-NBA value)
//   - If always uses 'ack':       req_cnt <= 0 in NBA  (too late for SVA)
//   - If always uses '$past(ack)': req_cnt unchanged in NBA
//
// CONCLUSION: the assertion sees the SAME req_cnt value (old accumulated
// count) regardless of whether 'ack' or '$past(ack)' is used for reset.
// The Preponed sampling protects the assertion from the NBA write.
//
// So: the sampling argument alone does NOT justify preferring $past(ack).
//
```

```
// === The REAL reasons $past(ack) is preferred ===
//
// Reason 1: multi-cycle ack correctness
//   If ack stays HIGH for N cycles, using 'ack' to reset req_cnt causes
//   it to clear to 0 on EVERY cycle while ack is high. This means a req
//   that rises during the ack window is not counted properly.
//   With $past(ack): resets exactly ONCE -- the cycle after ack rises.
//   Only the first ack cycle triggers the reset (next cycle $past(ack)=1,
//   the cycle after that $past(ack)=0 again).
//
// Reason 2: clean protocol boundary semantics
//   $past(ack) means: "the new counting window starts the cycle AFTER ack."
//   This models the protocol intent more precisely: ack ends the window,
//   and the next window begins on the following cycle.
//
// Reason 3: same-cycle req and ack behavior
//   If req rises on the same cycle as ack, with $past(ack) that req rise
//   is counted in the CURRENT window (not yet reset), and the counter
//   resets next cycle. This is typically the correct protocol behavior.
//
// Summary:
//   For a single-cycle ack pulse: both 'ack' and '$past(ack)' yield the
//   same assertion outcome (Preponed sampling ensures this).
//   For multi-cycle ack: $past(ack) is correct; 'ack' causes spurious resets.
//   Best practice: always use $past(ack) for clearer protocol semantics.
```

解释

这道题有一个值得深入思考的问题：

"用 ack 清零会导致 assertion 采样到 req_cnt = 0 吗？"

答案是：不会。这个担心本身是多余的。

SVA 在 Preponed region 采样信号，而 always @(posedge clk) 的赋值在 NBA 才生效，

Preponed 早于 NBA，所以 assertion 永远看到的是本拍 NBA 执行前的旧值。

无论 always 块里用 ack 还是 $past(ack) 来清零，

assertion 在 $rose(ack) 那拍看到的 req_cnt 都是累积的旧值，不受影响。

真正需要用 $past(ack) 的原因是协议行为的正确性，而非采样时机：

如果 ack 持续多拍，用 ack 清零会在 ack 高电平的每一拍都清零，

导致 ack 窗口期间新进来的 req 无法被正确计数。

$past(ack) 只清零一次（ack 拉高后的下一拍），语义更精确。

面试中能发现并正确解释这个采样时机的细节，是深刻理解 SV 调度模型的体现。

Q23. Gray Code: Verify single-bit change

QUESTION

Write code to verify that each Gray code transition changes

exactly 1 bit. Implement using both $onehot and $countones,

and write the corresponding SVA assertion.

ANSWER

```
logic [3:0] gray_prev, gray_curr;
logic valid;
```

```
// Method 1: $onehot - true if exactly one bit is 1
valid = $onehot(gray_prev ^ gray_curr);

// Method 2: $countones - count set bits
valid = ($countones(gray_prev ^ gray_curr) == 1);

// SVA assertion (checks every cycle)
property p_gray_valid;
  @(posedge clk) disable iff (rst)
  $onehot($past(gray_curr) ^ gray_curr);
endproperty
assert property (p_gray_valid);

// Note: $onehot0 returns true if AT MOST one bit is 1 (zero 1s OK)
// $onehot  returns true if EXACTLY one bit is 1 (zero 1s -> false)
```

解释

gray_prev XOR gray_curr 的结果中，只有发生变化的 bit 为1。

$onehot 检查一个向量中是否恰好只有一个 bit 为1，正好用于验证 Gray code。

$onehot0 则允许全0（即没有 bit 变化），与 $onehot 的区别面试常考。

SVA 版本用 $past(gray_curr) 获取上一拍的值，与当前值做 XOR，

在每个时钟上升沿验证只有一个 bit 变化。

## Q24. SVA: Five-Requirement Assertion Using throughout and within

QUESTION

Implement all 5 requirements in a single SVA assertion.

Use throughout and within operators.

Requirements:

1. A must rise first
2. B and C must rise after A rises
3. B and C must rise before A falls
4. C must fall before B falls
5. When A falls, B and C must already be 0

ANSWER

```
// Signal timeline:
// A: ____/^^^^^^^^^^^^^^^^^^^^\____
// B: _______/^^^^^^^^^^^^\__________   B rises after A, falls before A
// C: _________/^^^^^^^^^\____________   C rises after B, falls before B

// Provided correct assertion:
assert property (
  @(posedge clk) disable iff (!rst_n)
  $rose(A) |->
    (!B && !C) until_with (
      ($rose(B) ##[0:$] $rose(C))
      and
      (B throughout C[->1])
      ##1 $fell(B)
      ##[1:$] $fell(C)
```

```
      ##1 $fell(A)
    )
);

// Breakdown of each clause:
//
// $rose(A) |->
//   Trigger: fire when A has a rising edge.
//
// (!B && !C) until_with (...)
//   B and C must BOTH be LOW until (and including) the cycle where the
//   right-hand sequence starts matching. This enforces Req 1:
//   B and C cannot rise before A, and they start low.
//   until_with = strong version of until (RHS must eventually hold).
//
// $rose(B) ##[0:$] $rose(C)
//   B rises first (or simultaneously with C), then C rises.
//   Enforces Req 2: B and C must both rise.
//
// B throughout C[->1]
//   B stays HIGH from the point B rose until the NEXT occurrence of C
//   rising (C[->1] = next time C goes high). This ensures B is still
//   high when C rises -- B and C overlap. Enforces Req 3.
//
// ##1 $fell(B)
//   B falls exactly 1 cycle after the C[->1] reference point.
//
// ##[1:$] $fell(C)
//   C falls at least 1 cycle after B falls. Enforces Req 4:
//   C falls AFTER B (C outlasts B in the falling order).
//   NOTE: This means B falls first, then C falls -- which is the
//   order: both rise (B then C), then B falls, then C falls.
//
// ##1 $fell(A)
//   A falls exactly 1 cycle after C falls. At this point B and C are
//   already 0, so Req 5 is satisfied: when A falls, B and C are both 0.
```

解释

throughout 和 within 是 SVA 中处理复杂时序关系的关键操作符。

throughout：expr throughout seq 表示在整个 seq 序列匹配期间， expr 必须始终为真。

这里用来确保在 B 和 C 上升的整个等待期间 A 一直保持高电平。

within：seq1 within seq2 表示 seq1 的匹配必须在 seq2 的匹配窗口内发生。

这里用来确保 A 下降时 B 和 C 已经为 0。

这道题是面试中 SVA 的高难题，能写出骆架并解释每个操作符的作用就是加分项。

## SystemVerilog — Coding

### Q25. SV Coding: Monitor with Mailbox on Valid-Ready Interface

QUESTION

Write a SystemVerilog monitor class that:

- Observes a valid-ready handshake interface

- Captures a transaction when both valid and ready are HIGH on a rising clock edge
- Sends the captured transaction into a mailbox for the scoreboard or checker to consume

ANSWER

```
// Transaction item
class my_transaction;
  logic [31:0] data;
  logic [7:0]  addr;
endclass

// Monitor class
class my_monitor;
  // Mailbox shared with scoreboard (put from monitor, get from scbd)
  mailbox #(my_transaction) mbx;

  // Virtual interface handle
  virtual my_interface vif;

  function new(mailbox #(my_transaction) mbx,
               virtual my_interface vif);
    this.mbx = mbx;
    this.vif = vif;
  endfunction

  task run();
    my_transaction txn;
    forever begin
      // Wait for a clock edge where valid AND ready are both high
      @(posedge vif.clk);
      if (vif.valid && vif.ready) begin
        txn       = new();
        txn.data  = vif.data;
        txn.addr  = vif.addr;
        mbx.put(txn);  // non-blocking put; use put() to block if full
        $display("[MON] Captured txn: addr=%0h data=%0h",
                 txn.addr, txn.data);
      end
    end
  endtask

endclass

// UVM-style version using analysis port instead of mailbox:
class uvm_monitor extends uvm_monitor;
  `uvm_component_utils(uvm_monitor)
  uvm_analysis_port #(my_seq_item) ap;
  virtual my_interface vif;

  function void build_phase(uvm_phase phase);
    ap = new("ap", this);
    if (!uvm_config_db #(virtual my_interface)::get(
          this, "", "vif", vif))
      `uvm_fatal("NO_VIF", "")
  endfunction

  task run_phase(uvm_phase phase);
    my_seq_item item;
    forever begin
```

```
      @(posedge vif.clk);
      if (vif.valid && vif.ready) begin
        item      = my_seq_item::type_id::create("item");
        item.data = vif.data;
        item.addr = vif.addr;
        ap.write(item);  // broadcast to all connected subscribers
      end
    end
  endtask
endclass
```

解释

valid-ready 握手协议：两个信号同时为高时交易发生，是 AXI 等总线协议的基础。

Monitor 的职责：被动监射接口，不会驱动任何信号，只读不写。

Mailbox 所有权问题：通常 monitor 和 scoreboard 共享同一个 mailbox，

monitor put， scoreboard get，实现生产者-消费者模式。

UVM 风格中更推荐用 uvm_analysis_port + write()，

可以将捕获的 item 广播给多个订阅者（scoreboard、coverage collector 等）。

## UVM

### Q26. Virtual Sequencer vs Virtual Sequence

QUESTION

Compare Virtual Sequencer and Virtual Sequence across four

dimensions. Show with code how to implement concurrent

multi-agent stimulus coordination using virtual sequence.

Explain why $cast is needed.

ANSWER

```
Dimension        | Virtual Sequencer           | Virtual Sequence
-----------------|-----------------------------|-----------------------
Type             | uvm_sequencer (component)   | uvm_sequence (object)
Role             | Holds child seqr handles    | Coordinates stimulus
Contains         | References to real seqrs    | fork-join / seq logic
Instantiated in  | uvm_env                     | uvm_test or sequence

// Virtual sequencer
class my_virtual_sequencer extends uvm_sequencer;
  `uvm_component_utils(my_virtual_sequencer)
  my_sequencer agent_a_seqr;
  my_sequencer agent_b_seqr;
endclass

// Virtual sequence
class my_virtual_sequence extends uvm_sequence;
  `uvm_object_utils(my_virtual_sequence)
  task body();
    my_virtual_sequencer v_seqr;
```

```
    // $cast needed: p_sequencer is uvm_sequencer base type
    if (!$cast(v_seqr, p_sequencer))
      `uvm_fatal("CAST", "Virtual sequencer cast failed")
    fork
      seq_a.start(v_seqr.agent_a_seqr);
      seq_b.start(v_seqr.agent_b_seqr);
    join
  endtask
endclass
```

解释

Virtual sequencer 是一个组件（持久存在于 TB 层次结构中），

它不直接驱动任何接口，只是充当一个"中央控制台"，

保存所有 agent 内部真实 sequencer 的句柄。

Virtual sequence 是一个对象（每次测试创建并运行，运行完销毁），

通过 p_sequencer 获取 virtual sequencer 的引用，

然后在各个子 sequencer 上 fork 启动不同的子 sequence，

实现多 agent 的并发或顺序协调。

$cast 是必要的，因为 p_sequencer 的类型是基类 uvm_sequencer，

需要向下转型为 my_virtual_sequencer 才能访问子 sequencer 句柄。

## Q27. uvm_driver handshaking mechanism

QUESTION

Describe the complete request-response flow between

sequence -> sequencer -> driver -> DUT, including:

start_item, finish_item, get_next_item, item_done,

and the optional response path.

ANSWER

```
// Complete flow:
// 1. start_item(item): sequence calls wait_for_grant() to request
//    arbitration. Sequencer runs m_select_sequence() and grants
//    access to the winning sequence via wait_for_arbitration_completed().
//
// 2. finish_item(item): sequence randomizes item, then calls
//    send_request() to place item into m_req_fifo. Blocks until
//    driver calls item_done().
//
// 3. get_next_item(item): driver pulls item from m_req_fifo
//    and drives it to the DUT interface (pin-level).
//
// 4. item_done(): driver signals completion. Unblocks finish_item()
//    on the sequence side. Optionally puts a response.
//
// 5. get_response(item): sequence optionally retrieves the response
//    from response_queue, completing one full req-rsp cycle.

// Sequence side
task body();
  repeat(10) begin
```

```
    item = my_item::type_id::create("item");
    start_item(item);
    if (!item.randomize()) `uvm_fatal("RAND","")
    finish_item(item);       // blocks until item_done()
    // get_response(item);  // optional
  end
endtask

// Driver side
task run_phase(uvm_phase phase);
  forever begin
    seq_item_port.get_next_item(item);
    drive_item(item);            // pin-level drive
    seq_item_port.item_done();   // unblocks finish_item()
  end
endtask
```

解释

握手机制是 UVM 中最核心的数据流控制机制，面试必考。

关键点：start_item 负责仲裁（申请"谁的 sequence 可以发数据"），

finish_item 负责传递数据（把 item 放到 FIFO 里并等待 driver 取走）。

m_req_fifo 是一个 TLM FIFO，解耦了 sequence 和 driver 的时序。

item_done() 是握手的完成信号，driver 调用后 finish_item 才会解除阻塞，

sequence 才能发送下一个 item。

response 路径是可选的，driver 可以在 item_done() 的参数里附带响应，

sequence 调用 get_response() 取回。

## Q28. UVM Factory: how it works

QUESTION

Explain how the UVM Factory works:

(a) What are the four layers A::type_id::create("a") passes through?

(b) Difference between type override and instance override?

(c) Essential difference between type_id::create() and new()?

(d) Why does build phase go top-down and connect phase bottom-up?

ANSWER

```
// (a) Four layers of A::type_id::create("a"):
// Layer 1 - Class side (macro-generated):
//   type_id is typedef of uvm_object_registry#(A,"A")
//   calls T_create(name, parent, context)
//
// Layer 2 - Registry (uvm_object_registry):
//   calls factory.create_component_by_type(get(), context, name, parent)
//
// Layer 3 - Factory (uvm_default_factory):
//   resolves full instance path
//   calls find_override_by_type() to check type/instance overrides
//   calls resolved_type.create_object(name)
//
// Layer 4 - create_object:
```

```
//   calls new() on the resolved type, returns handle up the chain

// (b) Type override vs Instance override:
// Type override: replaces ALL instances of a type globally
my_driver::type_id::set_type_override(my_driver_v2::get_type());

// Instance override: replaces only a specific instance path
my_driver::type_id::set_inst_override(
  my_driver_v2::get_type(),
  "uvm_test_top.env.agent0.drv");

// (c) type_id::create() vs new():
//   new() bypasses factory -> no override possible
//   type_id::create() goes through factory -> override takes effect

// (d) build top-down: parent must exist before child can be created.
//     connect bottom-up: all children must exist before parent
//     can bind their TLM ports together.
```

解释

UVM Factory 的本质是一个带有覆盖机制的对象工厂。

type_id::create 和直接 new() 的本质区别：

new() 直接实例化，factory 完全不知道，无法被 override。

type_id::create 经过 factory，factory 会查找是否有 override，

如果有就创建 override 后的类型，这是 UVM 灵活性的核心。

type override 是全局替换，instance override 只替换特定路径的实例，

后者优先级更高。

build 自顶向下是因为子组件需要在父组件的 build_phase 中被 new() 出来，

必须先有父才能建子。

connect 自底向上是因为 TLM port 连接需要两端都已存在，

子组件的 port 连接好之后，父组件才能把子组件的 port 连接到更高层。

## Q29. Running the same sequence concurrently on multiple agents

QUESTION

You have the same sequence type and want to launch it on

multiple agents (sequencers) concurrently in uvm_test.

How do you do this? Why is fork/join_none the right choice here?

How do you manage objections correctly when using fork/join_none?

ANSWER

```
// Scenario: launch the same my_seq on 10 agents concurrently.

task run_phase(uvm_phase phase);
  phase.raise_objection(this);

  foreach (env.agent[i]) begin
    fork
      begin
```

```
        int idx = i;  // class methods are automatic by default
        my_seq seq;   // no 'automatic' keyword needed inside a class
        seq = my_seq::type_id::create($sformatf("seq_%0d", idx));
        seq.start(env.agent[idx].sequencer);
      end
    join_none  // fire and immediately move to next loop iteration
  end

  wait fork;  // block until ALL forked threads complete
  phase.drop_objection(this);
endtask

// Key SV storage class rules:
//
// Inside a CLASS method (task/function):
//   -> ALL local variables are automatic by default
//   -> 'automatic' keyword is redundant and unnecessary
//   -> Each call/fork gets its own private stack frame
//
// Inside a MODULE task/function:
//   -> Default is STATIC (all calls share the same variable)
//   -> Must declare 'automatic' explicitly to get per-call copies
//   -> OR declare the task as: task automatic my_task();
//
// Why begin..end is still needed here:
//   Variable declarations must be inside a sequential block.
//   They cannot float bare between fork..join_none.
//   begin..end creates the required block scope for idx and seq.
//
// Classic exam pattern (CORRECT, inside a class):
//   fork
//     begin
//       int idx = i;   // automatic by default in class context
//       my_seq seq;
//       seq.start(env.agent[idx].sequencer);
//     end
//   join_none
//
// Common bug (shares loop variable 'i' directly):
//   foreach (env.agent[i]) begin
//     fork
//       begin
//         seq.start(env.agent[i].sequencer);  // BUG: 'i' is the
//       end                  // loop var, shared across all threads
//     join_none              // all threads see i=N when they run
//   end

// Why fork/join_none?
// fork/join     -> blocks until thread finishes -> serializes loop
// fork/join_any -> same problem, blocks the loop iteration
// fork/join_none-> returns immediately -> all N threads launch
//                  before any one of them finishes -> TRUE concurrency

// Why wait fork + objection pattern?
// fork/join_none does not wait. Without 'wait fork', drop_objection
// executes while sequences are still running -> simulation ends early.
// 'wait fork' ensures all background threads complete first.
```

解释

这道题的核心场景是：同一个 sequence 类型，要并发跑在多个 agent 上。

为什么 class 内部不需要 automatic 关键字：

SV 规定 class 内部的 task/function，所有局部变量默认就是 automatic 存储类型。

每次 fork 都会在自己的栈帧上创建独立的变量副本，不需要显式声明 automatic。

automatic 关键字只在 module 内部的 task/function 中才需要显式声明，

因为 module 里默认是 static（所有调用共享同一个变量）。

uvm_test 的 run_phase 是 class method，所以 idx 和 seq 声明 int/my_seq 即可。

为什么还需要 begin..end：

SV 语法规定变量声明必须在顺序块（begin..end）内部，

不能直接裸露在 fork..join_none 之间，那是语法错误。

begin..end 创建了独立的块作用域，每次 fork 启动时

在该线程的栈帧上创建 idx 和 seq 各自的副本。

为什么不能直接用循环变量 i（最常考的陷阱）：

如果直接写 seq.start(env.agent[i].sequencer)，

i 是 foreach 的循环变量，声明在 begin..end 外部。

所有线程共享这同一个 i，循环跑完时 i 已经等于 N，

所有线程都去访问 env.agent[N]，越界或全部访问同一个错误的 agent。

把 i 赋值给 begin..end 内部的 idx，每个线程在自己的栈帧上保存了正确的值。

为什么是 fork/join_none：

fork/join 和 fork/join_any 在循环里会阻塞循环体，变成串行执行。

fork/join_none 立即返回，循环快速启动所有 agent 的 seq，全部并发运行。

wait fork 与 objection：

fork/join_none 后程序立即往下走，必须用 wait fork 阻塞，

等待所有线程完成后再 drop_objection，否则仿真过早结束。

## Q30. uvm_config_db: how it works

QUESTION

Explain how uvm_config_db works. Cover:

(a) Meaning of the 4 parameters in set/get

(b) Difference between null context and a component context

(c) How to use $sformatf to configure multiple agents dynamically

(d) Wildcard path matching

(e) default_sequence mechanism and uvm_object_wrapper

ANSWER

```
// Syntax:
// uvm_config_db #(TYPE)::set(context, inst_path, field_name, value);
// uvm_config_db #(TYPE)::get(context, inst_path, field_name, variable);

// (a) Parameters:
//   context   : the calling component (path resolution root)
//               null means root is uvm_test_top
```

```
//   inst_path : relative path from context to target component
//               supports wildcards (*)
//   field_name: key string (must match exactly between set and get)
//   value     : the value to store or retrieve

// (b) null context vs component context:
//   null   -> full path must start from uvm_test_top
//   this   -> path is relative to current component's full path

// Passing virtual interface (from top-level module):
uvm_config_db #(virtual my_if)::set(
  null, "uvm_test_top.env.agent.drv", "vif", my_if);

// (c) $sformatf for dynamic paths:
for (int i = 0; i < 10; i++)
  uvm_config_db #(uvm_object_wrapper)::set(
    this,
    $sformatf("env.agent%0d.sequencer", i),
    "default_sequence",
    my_seq::get_type());

// (d) Wildcard path:
uvm_config_db #(uvm_object_wrapper)::set(
  this, "env.agent*.sequencer",
  "default_sequence", my_seq::get_type());

// (e) default_sequence: sequencer automatically starts this sequence
//     in run_phase without explicit seq.start() in the test.
//     uvm_object_wrapper is used to pass a TYPE (not an instance).
```

解释

uvm_config_db 是一个全局键值数据库，key 由 context + inst_path + field_name

三部分组合而成。set 和 get 的 field_name 必须完全一致才能匹配。

null context 等价于以 uvm_test_top 为根节点的绝对路径。

$sformatf 是动态构建字符串路径的标准做法，用于批量配置多个 agent。

通配符 * 可以匹配任意字符串，方便一次性配置所有 agent。

default_sequence 机制让 sequencer 在 run_phase 自动启动配置好的 sequence，

不需要在 test 里手动调用 seq.start()，传入的是 get_type() 返回的类型包装器。

## Q31. What happens inside UVM after run_test() is called

QUESTION

Describe the complete sequence of events after run_test() is called:

(a) How +UVM_TESTNAME is read and the test class instantiated

(b) Creation of uvm_test_top root node

(c) Full phase execution order

(d) How objection mechanism controls run_phase termination

(e) When TLM port binding occurs

ANSWER

```
// Step 1: uvm_coreservice + uvm_cmdline_processor
```

```
//   Read +UVM_TESTNAME from command line (or use hardcoded name)
//   Factory instantiates the test class -> creates uvm_test_top
//   as the root component under uvm_root

// Step 2: uvm_root::run_test
//   Initializes objection mechanism
//   Calls m_run_phases to begin phase execution

// Step 3: Phase execution order (12 phases):
//   build_phase          -> top-down (parent creates children)
//   connect_phase        -> bottom-up (TLM port connections)
//   start_of_simulation  -> bottom-up (print topology etc.)
//   run_phase            -> concurrent (all components in parallel)
//   [12 run-time phases: pre_reset -> reset -> ... -> shutdown]
//   extract_phase        -> bottom-up
//   check_phase          -> bottom-up
//   report_phase         -> bottom-up
//   final_phase          -> top-down

// Step 4: TLM port binding
//   phase_started callback triggers uvm_component::do_resolve_bindings
//   -> uvm_port_base::resolve_bindings
//   All TLM ports are connected before run_phase begins

// Step 5: run_phase + objection
//   Components raise objections to keep run_phase alive
//   When all objections dropped: all_dropped -> phase ends
//   Simulator exits after final_phase completes
```

解释

run_test() 是把控制权从 SystemVerilog 调度器移交给 UVM phasing 引擎的唯一入口。

+UVM_TESTNAME 允许在命令行动态指定测试类名，不需要重新编译。

build 自顶向下的关键原因：父组件在 build_phase 里用 type_id::create 建子组件，

子组件必须等父组件 build 完才存在。

connect 自底向上：子组件的 TLM port 先连好，父组件再把子组件的 port 串联。

run_phase 是并发的：所有组件的 run_phase 在同一时刻开始，互相独立运行。

objection 机制是仿真结束的"投票"系统：

任何一个 raise 没有对应 drop，仿真就不会结束。

## Q32. Write a simplified config_db class from scratch

QUESTION

Write a simplified config_db class in SystemVerilog:

- Parameterized to support any type
- Static set(path, field, value) using concatenated key
- Static get(path, field, ref value) returning success bit
- Internal storage using associative array

ANSWER

```
class config_db #(type T = int);
  static T db[string];  // associative array: key -> value
```

```
  static function void set(
      string path, string field, T value);
    db[{path, ".", field}] = value;
  endfunction

  static function bit get(
      string path, string field, ref T value);
    string key = {path, ".", field};
    if (db.exists(key)) begin
      value = db[key];
      return 1;
    end
    return 0;
  endfunction
endclass

// Usage:
config_db #(int)::set("env.agent0", "depth", 8);
int val;
if (config_db #(int)::get("env.agent0", "depth", val))
  $display("got: %0d", val);  // prints: got: 8
```

解释

这道题考查 SV OOP 中参数化类（parameterized class）和静态成员的使用。

关键点：

1. #(type T = int) 使这个类可以存储任意类型，T 是类型参数，默认 int。

2. static 修饰的成员变量和方法属于类本身，不属于某个实例，

可以直接用 config_db#(int)::set() 调用，不需要 new()。

3. 关联数组 db[string] 用字符串作为 key，存储任意类型的值。

4. key = {path, ".", field} 把两个路径段拼接成唯一 key，

模拟真实 UVM config_db 的路径匹配逻辑。

5. get 返回 bit 表示是否找到，通过 ref 参数输出值。

## Q33. UVM Coding: AHB Driver with Pipelined Address/Data Phases

QUESTION

Write a UVM driver for the AHB (Advanced High-performance Bus) protocol.

AHB has a pipelined two-phase structure:

- Address Phase: drive HADDR, HWRITE, HSIZE for the CURRENT transaction

- Data Phase : drive HWDATA for the PREVIOUS transaction

(data phase of txn N overlaps with address phase of txn N+1)

The driver must:

1. In the first cycle of each transaction, drive the current transaction's

address signals (HADDR, HWRITE, HSIZE) -- Address Phase

2. Simultaneously drive the previous transaction's write data (HWDATA)

-- Data Phase of the previous transaction

3. Handle the very first transaction correctly (no previous data to drive)

4. After the last transaction, complete the final data phase

AHB pipelined timing:

Cycle: | 1 | 2 | 3 | 4 |

HADDR: | A0 | A1 | A2 | --- |

HWDATA: | --- | D0 | D1 | D2 |

Note: D0 (data for txn 0) is driven in cycle 2, same cycle as A1

ANSWER

```
// AHB transaction item
class ahb_seq_item extends uvm_sequence_item;
  `uvm_object_utils(ahb_seq_item)
  rand logic [31:0] haddr;
  rand logic [31:0] hwdata;
  rand logic        hwrite;
  rand logic [2:0]  hsize;
endclass

// AHB Driver
class ahb_driver extends uvm_driver #(ahb_seq_item);
  `uvm_component_utils(ahb_driver)

  virtual ahb_interface vif;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(virtual ahb_interface)::get(
          this, "", "vif", vif))
      `uvm_fatal("NO_VIF", "AHB virtual interface not found")
  endfunction

  task run_phase(uvm_phase phase);
    ahb_seq_item curr_item;
    ahb_seq_item prev_item = null;  // holds previous transaction

    // Initialize bus to IDLE state
    drive_idle();

    forever begin
      // ■■ Address Phase: fetch and drive current transaction ■■
      seq_item_port.get_next_item(curr_item);

      @(posedge vif.clk);

      // Drive current transaction's ADDRESS phase
      vif.haddr  <= curr_item.haddr;
      vif.hwrite <= curr_item.hwrite;
      vif.hsize  <= curr_item.hsize;
      vif.htrans <= 2'b10;  // NONSEQ: start of new transfer

      // Simultaneously drive previous transaction's DATA phase
      if (prev_item != null && prev_item.hwrite)
        vif.hwdata <= prev_item.hwdata;

      seq_item_port.item_done();

      // Save current as previous for next iteration
      prev_item = curr_item;
    end
  endtask
```

```
  // Call after the forever loop exits (e.g. in a wrap-up task)
  // to complete the final transaction's data phase
  task complete_last_data_phase(ahb_seq_item last_item);
    @(posedge vif.clk);
    if (last_item != null && last_item.hwrite)
      vif.hwdata <= last_item.hwdata;
    vif.htrans <= 2'b00;  // IDLE
  endtask

  task drive_idle();
    @(posedge vif.clk);
    vif.htrans <= 2'b00;  // IDLE
    vif.haddr  <= '0;
    vif.hwdata <= '0;
    vif.hwrite <= 1'b0;
  endtask

endclass

// Cycle-by-cycle trace for 3 write transactions:
//
// Cycle 1: get txn0 -> drive HADDR=A0, HWRITE=1  | HWDATA=X (no prev)
// Cycle 2: get txn1 -> drive HADDR=A1, HWRITE=1  | HWDATA=D0 (prev=txn0)
// Cycle 3: get txn2 -> drive HADDR=A2, HWRITE=1  | HWDATA=D1 (prev=txn1)
// Cycle 4: complete -> drive HTRANS=IDLE          | HWDATA=D2 (prev=txn2)
```

解释

AHB 最关键的特性是 Pipeline 结构：

Address Phase 和 Data Phase 不是同一笔 transaction 的前后两个周期，

而是第 N 笔 transaction 的 Data Phase 和第 N+1 笔的 Address Phase 重叠在同一个周期。

这意味着 driver 在每个周期要同时驱动两笔 transaction 的信号：

- 当前答的地址（当前笔 addr phase）

- 上一答的数据（上一笔 data phase）

实现核心思路：用 prev_item 保存上一笔 transaction。

每次循环：

1. get_next_item 取当前笔

2. 驱动当前笔的 HADDR/HWRITE/HSIZE（address phase）

3. 驱动 prev_item 的 HWDATA（data phase）

4. 把当前笔存为 prev_item，等待下一循环

两个边界条件需要特别处理：

1. 第一笔 transaction：prev_item = null，不驱动 HWDATA（或驱动为0）。

2. 最后一笔 transaction：循环结束后还要再多驱动一个周期来完成最后一笔的 data phase。

为什么需要 item_done() 在 @posedge clk 前调用？

实际上 item_done() 在驱动信号之后、下个 get_next_item 之前调用即可。

这里在 @posedge clk 之后、保存 prev_item 之前调用，

确保 sequence 在 driver 完成当前周期驱动后才能发下一笔。

面试重点：能说清楚 AHB pipeline 为什么需要 prev_item，

以及第一笔和最后一笔边界处理的逐层逻辑。

## Q34. UVM: Shallow Copy vs Deep Copy and Their Applications in Testbench

QUESTION

Explain the difference between shallow copy and deep copy in SystemVerilog/UVM. For each, describe:

(a) How it is implemented in SV and via UVM built-in methods

(b) What gets copied and what gets shared

(c) A concrete UVM testbench scenario where using the wrong copy type causes a bug

(d) When to use each in practice

ANSWER

```
// ■■ Data structure used in examples ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class sub_payload extends uvm_object;
  `uvm_object_utils(sub_payload)
  rand byte data[];       // dynamic array inside nested object
endclass

class my_seq_item extends uvm_sequence_item;
  `uvm_object_utils(my_seq_item)
  rand logic [31:0] addr;
  rand logic [31:0] data;
  sub_payload        payload;  // handle to a nested object
endclass

// ■■ Shallow Copy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Method 1: SV copy constructor
my_seq_item a = new();
my_seq_item b = new a;   // shallow copy

// Method 2: UVM built-in  (calls do_copy internally)
my_seq_item b;
b = my_seq_item::type_id::create("b");
b.copy(a);               // shallow copy by default unless do_copy overridden

// What gets copied:
//   b.addr    = a.addr    -> independent (value type, safe)
//   b.data    = a.data    -> independent (value type, safe)
//   b.payload = a.payload -> SAME object! (handle copy, SHARED)
//
// Bug scenario (scoreboard):
//   Monitor captures item, pushes to scoreboard queue.
//   Driver calls item.randomize() for the next transaction --
//   this re-randomizes payload.data[] IN PLACE.
//   Scoreboard now sees the modified payload -> comparison fails
//   even though DUT output was correct. False failure.

// ■■ Deep Copy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Must override do_copy() to manually deep-copy nested objects:
class my_seq_item extends uvm_sequence_item;
  `uvm_object_utils(my_seq_item)
  rand logic [31:0] addr;
  rand logic [31:0] data;
  sub_payload        payload;
```

```
  function void do_copy(uvm_object rhs);
    my_seq_item rhs_;
    super.do_copy(rhs);                       // copy base class fields
    if (!$cast(rhs_, rhs))
      `uvm_fatal("CAST", "do_copy cast failed")
    addr    = rhs_.addr;
    data    = rhs_.data;
    // Deep copy: create a NEW sub_payload and copy its contents
    payload = sub_payload::type_id::create("payload");
    payload.copy(rhs_.payload);               // recursive deep copy
  endfunction
endclass

// After deep copy:
//   b.addr    = a.addr    -> independent
//   b.data    = a.data    -> independent
//   b.payload             -> NEW object, independent from a.payload
//   Modifying a.payload does NOT affect b.payload

// ■■ UVM clone() = create() + copy() ■■■■■■■■■■■■■■■■■■■■■■■■■■
// Convenience method that allocates a new object and deep-copies:
my_seq_item b;
$cast(b, a.clone());
// Equivalent to:
//   b = my_seq_item::type_id::create("b");
//   b.copy(a);

// ■■ Practical guidelines ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Use shallow copy when:
//   - seq_item has NO nested class handles (only integral fields)
//   - temporary local reference, object will not be modified
//   - performance-critical path, nested objects are large/expensive
//
// Use deep copy (override do_copy) when:
//   - seq_item contains nested uvm_object handles (sub-objects)
//   - item is stored in scoreboard / reference model for later compare
//   - item is reused across multiple transactions (avoid aliasing)
//   - monitor clones item before pushing to analysis port
```

解释

浅拷贝和深拷贝的本质区别：

浅拷贝只拷贝对象的最外一层，对于内嵌的 class 句柄，

只复制句柄本身，不复制句柄指向的对象。

深拷贝则递归地为每个嵌套对象都分配新内存并复制内容，

复制后所有对象完全独立。

UVM TB 中最常见的错误场景：

Monitor 损取到一个 seq_item 后，用浅拷贝把它存入 scoreboard 队列。

如果这个 item 里有嵌套对象（比如 payload），

scoreboard 里存的句柄和原 item 指向同一个 payload。

Driver 在下一笔 transaction 重新随机化时， payload 被修改，

scoreboard 看到的参考数据也跟着变了，导致误判。

这种 bug 非常难排查，因为 DUT 输出是对的，错的是 TB 自身的拷贝逻辑。

UVM 的 do_copy() 设计思路：

UVM 的 copy() 方法内部调用 do_copy()，

默认实现是浅拷贝（只复制基类定义的字段）。

要实现深拷贝，必须在子类中重写 do_copy()，

手动为每个嵌套对象调用 create() + copy()。

clone() 是 UVM 提供的便捷方法：

内部等价于 type_id::create() + copy()，

返回一个新对象，是 monitor 向 analysis port 写入数据时的推荐写法。

实际项目中的选择原则：

如果 seq_item 只有 int/logic 等内建类型字段，浅拷贝安全且性能更好。

只要 seq_item 包含任何 class 句柄字段，就必须重写 do_copy() 实现深拷贝，

否则故障难以排查。

## Digital Design

### Q35. Gray Code: Encode and decode

QUESTION

Implement 4-bit Binary-to-Gray and Gray-to-Binary conversion

functions. Explain why Gray code is used for async FIFO pointers

and what advantage it provides.

ANSWER

```
// Binary -> Gray: G[i] = B[i] XOR B[i+1], MSB: G[N-1] = B[N-1]
function automatic logic [3:0] bin2gray(logic [3:0] b);
  return b ^ (b >> 1);
endfunction

// Gray -> Binary: B[MSB] = G[MSB]; B[i] = B[i+1] XOR G[i]
function automatic logic [3:0] gray2bin(logic [3:0] g);
  logic [3:0] b;
  b[3] = g[3];
  b[2] = b[3] ^ g[2];
  b[1] = b[2] ^ g[1];
  b[0] = b[1] ^ g[0];
  return b;
endfunction

// Why Gray code for async FIFO pointers?
// Binary counter: 0111 -> 1000 changes 4 bits simultaneously.
// If sampled mid-transition across clock domains, any intermediate
// value (e.g. 1111, 1001) could be captured -> catastrophic error.
// Gray code: only 1 bit changes per increment.
// A metastability event on that 1 bit causes at most an off-by-one
// error, which is safe for full/empty detection.
```

解释

二进制计数器在进位时可能同时翻转多个 bit（比如 0111->1000 翻转4个bit），

如果在跨时钟域采样时恰好捕捉到中间状态，读到的值可能是完全错误的数字。

Gray code 保证相邻值之间只有1个 bit 变化，即使在亚稳态情况下采到的值

只会是"前一个"或"后一个"，最多产生 off-by-one 的误差，

对于 FIFO 的空满判断来说是安全的（顶多误判一次，不会读出完全错误的地址）。

bin2gray 公式 b ^ (b >> 1) 非常简洁，是硬件面试的必考点。

gray2bin 需要从 MSB 逐位向下推导，因为每一位依赖高位的 Binary 值。

## Q36. Clock Domain Crossing (CDC) techniques

QUESTION

What are the common CDC techniques? Explain each one's applicable

scenario and working principle:

Two-FF Synchronizer, Handshake Protocol, Async FIFO,

Gray code pointer encoding, MUX/Enable Synchronizer.

ANSWER

```
Technique              | Use Case                  | Key Principle
-----------------------|---------------------------|-------------------
Two-FF Synchronizer    | Single-bit control signal | 2 flops reduce MTBF
Handshake Protocol     | Multi-bit control/data    | req/ack confirmation
Async FIFO             | Continuous data stream    | Gray-coded pointers
Gray Code Pointers     | FIFO full/empty detect    | 1-bit change only
MUX/Enable Synchronizer| Mux select across domains | Synchronize enable

// Two-FF: reduces probability of metastability propagating
always_ff @(posedge clk_dst) begin
  sync_ff1 <= async_signal;
  sync_ff2 <= sync_ff1;  // use sync_ff2 as output
end

// Handshake: ensures multi-bit data is stable when sampled
// Source domain pulses req; destination samples data only after
// its own synchronized version of req is seen; ack confirms receipt.

// Async FIFO: write pointer in write domain, read pointer in read
// domain. Both are Gray-coded before crossing to the other domain.
```

解释

CDC 是芯片设计中最容易出 bug 且最难调试的问题之一。

双触发器同步器：一个触发器出现亚稳态的概率很高，经过第二个触发器

亚稳态分辨出来的概率大幅提升（MTBF 指数级增长）。

握手协议：适用于不能用 FIFO 的场景，用 req/ack 四段握手确保数据稳定采样，

但吞吐量较低（每次传输需要4个半周期）。

异步 FIFO：最通用的解决方案，支持连续数据流，核心难点在于

读写指针要用 Gray code 编码后再跨时钟域传递。

面试重点：为什么用 Gray code？因为二进制计数器进位时多 bit 同时翻转，

Gray code 保证每次只变一个 bit，大大降低 CDC 采样错误的风险。

## Q37. Moore vs Mealy state machine

QUESTION

Compare Moore and Mealy state machines across four dimensions:

output dependency, state count, area, and output timing/glitch risk.

Which one has larger area and why?

ANSWER

```
Dimension       | Moore                         | Mealy
----------------|-------------------------------|---------------------
Output depends  | Current state only            | State + current input
State count     | Usually MORE states           | Usually FEWER states
Area            | Usually LARGER                | Usually smaller
Output timing   | Registered (1 cycle latency) | Combinational (immediate)
Glitch risk     | Low (output from registers)   | Higher (input-dependent)

// Why Moore is usually larger?
// Moore must encode the output in the state itself.
// Different output combinations require different states.
// Mealy can share states and differentiate output via input,
// reducing total state count and thus state register bits.

// Example: sequence detector "101"
// Moore: needs 4 states (S0,S1,S2,S3) + separate output logic
// Mealy: needs 3 states, output asserted on S2 when input=1
```

解释

Moore 机的输出只依赖当前状态，所以输出是寄存器输出，

延迟一个时钟周期但非常干净，不受输入毛刺影响。

Mealy 机的输出依赖状态和当前输入，所以是组合逻辑输出，

响应更快（同一拍就能输出），但输入上的毛刺会直接传播到输出。

面积比较：Moore 通常面积更大，因为需要更多状态来编码所有输出组合，

而 Mealy 可以用更少状态，靠输入区分不同输出。

实际设计中 Moore 更常用，因为输出是寄存器驱动，时序更好分析。

面试重点：Moore 面积大、时序好、无毛刺；Mealy 面积小、响应快、有毛刺风险。

## Q38. Timing issues and solutions

QUESTION

What are common timing issues in digital circuits?

For each, explain the root cause and corresponding solution:

Setup Violation, Hold Violation, Clock Skew,

Metastability, Glitch.

ANSWER

```
Issue           | Root Cause                     | Solution
----------------|--------------------------------|---------------------
Setup Violation | Data arrives too late before   | Reduce logic depth,
```

```
                | clock edge (long data path)    | pipeline, reduce freq
Hold Violation  | Data changes too soon after    | Insert delay buffers,
                | clock edge (short data path)   | fix clock skew
Clock Skew      | Clock arrives at different     | CTS (Clock Tree
                | times at different FFs         | Synthesis), balancing
Metastability   | Async signal sampled in        | Two-FF synchronizer,
                | setup/hold window              | increase MTBF margin
Glitch          | Multiple paths with different  | Register outputs,
                | delays (combinational hazard)  | add timing constraints

// Setup: data must be stable BEFORE clock edge by at least t_setup
// Hold:  data must remain stable AFTER clock edge for at least t_hold
// These are independent: fixing setup can worsen hold and vice versa
```

解释

Setup 和 Hold 是最基础的时序概念，但经常混淆：

Setup violation：数据路径太慢，时钟来了数据还没准备好。

解决：减少数据路径的逻辑深度（插流水线寄存器）、降低时钟频率、提高电压。

Hold violation：数据路径太快，时钟来了数据已经翻走了。

解决：在数据路径上加延迟（buffer）、减小时钟偏差。

关键认识：Setup 和 Hold 是互相独立的约束，

插入 buffer 解决 hold 不会影响 setup（因为 setup 约束在下一个时钟周期）。

Metastability 无法完全消除，只能用双触发器降低传播概率（提高 MTBF）。

Clock skew 过大会导致 hold violation（launch FF 的时钟到得快，

capture FF 的时钟到得慢，hold 窗口变小）。

## Q39. FSM design: output 1 when serial input (LSB first) is divisible by 5

QUESTION

Design an FSM: bits are input serially starting from LSB.

Output 1 when the value formed by all received bits so far

is divisible by 5. Draw the state transition diagram

(5 states, corresponding to remainders 0-4) and write RTL.

ANSWER

```
// Key insight: when new bit b is shifted in from LSB,
// current value N becomes 2*N + b
// new_remainder = (old_remainder * 2 + b) % 5

// State transition table:
// State (remainder) | input=0 -> next | input=1 -> next | output
// S0 (rem=0)        |   S0            |   S1            |   1
// S1 (rem=1)        |   S2            |   S3            |   0
// S2 (rem=2)        |   S4            |   S0            |   0
// S3 (rem=3)        |   S1            |   S2            |   0
// S4 (rem=4)        |   S3            |   S4            |   0

typedef enum logic [2:0] {S0,S1,S2,S3,S4} state_t;
state_t state, next_state;
```

```
always_ff @(posedge clk or posedge rst)
  if (rst) state <= S0;
  else     state <= next_state;

always_comb begin
  case (state)
    S0: next_state = bit_in ? S1 : S0;
    S1: next_state = bit_in ? S3 : S2;
    S2: next_state = bit_in ? S0 : S4;
    S3: next_state = bit_in ? S2 : S1;
    S4: next_state = bit_in ? S4 : S3;
    default: next_state = S0;
  endcase
end

assign out = (state == S0);  // Moore output
```

解释

这道题的关键是推导状态转移方程。

当新 bit 从 LSB 输入时，已有的数 N 变为 2N + bit_in（因为新 bit 是最低位）。

所以新余数 = (旧余数 × 2 + bit_in) % 5，据此可以推导出所有状态转移。

以 S2（余数2）为例：

输入0：(2×2+0)%5 = 4 -> S4

输入1：(2×2+1)%5 = 0 -> S0

这是一个 Moore 机，输出只在 S0（余数为0）时为1。

面试重点：能快速推导状态转移表，然后写出标准的三段式 FSM RTL。

## Q40. FIFO depth calculation

QUESTION

Given: f_tx = 4 * f_rx, period(wen) = 100 * period(f_rx),

wen duty cycle = 25%.

Calculate the minimum FIFO depth to avoid overflow and underflow.

Show step-by-step calculation.

ANSWER

```
// Step 1: Data written per wen period
//   wen high duration = 25% x 100 x period(f_rx)
//                     = 25 x period(f_rx)
//   Write clock period = period(f_rx) / 4
//   Data written = 25 x period(f_rx) / (period(f_rx)/4)
//                = 25 x 4 = 100 words

// Step 2: Data read per wen period
//   Read clock = f_rx, reading continuously
//   Data read = 100 x period(f_rx) / period(f_rx) = 100 words

// Step 3: Maximum accumulation (worst case burst)
//   During wen high (25 f_rx cycles):
//     Written: 25 x 4 = 100 words  (write rate = 4x read rate)
//     Read:    25 x 1 = 25 words
```

```
//     Net accumulation = 100 - 25 = 75 words

// Minimum FIFO depth = 75

// Formula shortcut:
// Depth = (write_rate - read_rate) x wen_high_duration
//       = (4 - 1) x 25 = 75
```

解释

FIFO 深度计算的核心思路：找出最坏情况下 FIFO 里最多会积累多少数据。

最坏情况就是 wen 高电平期间（突发写入阶段）：

- 写速率是读速率的4倍（f_tx = 4*f_rx）
- wen 高电平持续 25 个 f_rx 周期
- 这段时间写入了 100 个数据，但只读走了 25 个
- 净积累 = 75 个数据

wen 低电平期间：不写入但继续读，积累量只会减少，不是瓶颈。

所以 FIFO 最小深度为 75，保证突发写入时不会 overflow，

读速率持续所以不存在 underflow 风险。

## Q41. Roofline Model analysis

QUESTION

Platform: Peak Compute = 16 TFLOPS, Peak Memory BW = 512 GB/s,

float32 (4 Bytes per element).

For matrix multiplication C = A x B at three sizes,

determine arithmetic intensity and bound type for each case.

Cases: 64x64, 512x512, 4096x4096.

ANSWER

```
// General formulas for N x N matmul:
//   FLOPs  = 2 * N^3    (N^2 outputs, each needs N MACs)
//   Memory = 3 * N^2 * 4 Bytes  (read A, read B, write C)
//   Arithmetic Intensity (AI) = FLOPs / Memory
//                             = 2*N^3 / (12*N^2) = N/6  [FLOP/Byte]

// Ridge Point = Peak Compute / Peak BW
//             = 16 TFLOPS / 512 GB/s = 31.25 FLOP/Byte

// Case 1: N=64
//   AI = 64/6 ≈ 10.7 FLOP/Byte  < 31.25 -> MEMORY-BOUND
//   Attainable = AI x BW = 10.7 x 512 ≈ 5.5 TFLOPS

// Case 2: N=512
//   AI = 512/6 ≈ 85.3 FLOP/Byte > 31.25 -> COMPUTE-BOUND
//   Attainable = Peak Compute = 16 TFLOPS

// Case 3: N=4096
//   AI = 4096/6 ≈ 682 FLOP/Byte >> 31.25 -> COMPUTE-BOUND
//   Attainable = Peak Compute = 16 TFLOPS
```

```
// Conclusion: as matrix size increases, AI increases proportionally
// to N. Small matrices are memory-bound (low hardware utilization).
// Large matrices are compute-bound (near peak utilization).
```

解释

Roofline Model 是分析计算性能瓶颈的经典工具。

算术强度（Arithmetic Intensity）= 计算量 / 内存访问量，

单位是 FLOP/Byte，表示每读写一个 Byte 的数据能做多少计算。

Ridge Point 是分界点：AI 低于 Ridge Point 时受内存带宽限制（memory-bound），

AI 高于 Ridge Point 时受计算能力限制（compute-bound）。

矩阵乘法中 AI = N/6，矩阵越大 AI 越高，越能充分利用计算资源。

小矩阵（N=64）：大部分时间在等内存数据，GPU 利用率很低。

大矩阵（N=4096）：内存带宽不是瓶颈，计算单元接近满负荷。

实际优化中：memory-bound 问题用 data reuse / tiling / cache 优化；

compute-bound 问题用算法优化（减少 FLOPs）或更快的计算单元。

## Q42. Two types of power and how to optimize

QUESTION

What are the two main types of power in digital circuits?

For each, explain: source, formula, optimization techniques.

How do you choose the optimization approach based on scenario?

ANSWER

```
// Type 1: Dynamic Power (switching power)
//   Source: capacitor charging/discharging during signal transitions
//   Formula: P_dynamic = a * C * V^2 * f
//     a = activity factor (switching probability)
//     C = load capacitance
//     V = supply voltage
//     f = clock frequency
//   Dominant in: active, high-frequency operation

// Type 2: Static Power (leakage power)
//   Source: sub-threshold leakage current even when circuit is idle
//   Formula: P_static = I_leak * V_dd
//   Dominant in: idle/standby mode, advanced process nodes (< 28nm)

// Dynamic Power Optimization:
//   Clock Gating    : disable clock to idle registers -> reduces a
//   Operand Isolation: gate inputs to idle logic -> reduces a
//   DVFS            : lower V and f when full perf not needed
//                     V^2 term makes this very effective
//   Multi-Vdd       : use lower voltage for non-critical logic
//   Data encoding   : Gray code for buses reduces toggle rate

// Static Power Optimization:
//   Multi-Vt cells  : HVT (high threshold) = low leakage, slow
//                     LVT (low threshold)  = high leakage, fast
//                     Use HVT on non-critical paths
```

```
//   Power Gating    : completely shut off power to idle blocks
//                     requires retention cells to save state
//   Body Biasing    : Reverse Body Bias raises Vt, reduces leakage
//                     Forward Body Bias lowers Vt, increases speed

// How to choose:
//   High-performance mode : focus on Dynamic Power
//     -> Clock gating, DVFS, reduce activity factor
//   Standby/sleep mode    : focus on Static Power
//     -> Power gating, HVT cells, reverse body bias
//   PPA trade-off: LVT = fast but leaky; HVT = slow but efficient
//     Critical path: LVT; non-critical: HVT for leakage saving
```

解释

Dynamic Power 的公式 $P = \alpha CV^2f$ 中，电压是平方项，影响最大。

DVFS（动态电压频率调节）是最有效的 dynamic power 优化手段，

频率降低一半，功耗约减少一半；电压降低一半，功耗减少四分之三。

Clock Gating 是芯片设计中最常用的节能技术，

通过在寄存器时钟端插入 enable 门控，让不需要更新的寄存器停止翻转。

Static Power 在先进工艺（28nm 以下）中越来越显著，

因为晶体管尺寸缩小，Vt 降低，亚阈值漏电流增大。

Power Gating 是最激进的静态功耗优化，直接关掉模块的电源，

但需要 retention flip-flop 来保存状态，唤醒时间也较长。

Multi-Vt 选择是 PPA 权衡的典型体现：关键路径用 LVT 保证时序，

非关键路径用 HVT 减少漏电，通常可以在不影响性能的前提下显著降低功耗。

## Q43. Fixing setup/hold violations after tape-out

QUESTION

After tape-out, if setup time or hold time violations are found,

RTL and physical implementation cannot be changed.

(a) What are the post-tape-out options for setup violations?

(b) What are the post-tape-out options for hold violations?

(c) What is the fundamental difference in fixability between

setup and hold violations after tape-out?

(d) How can ECO prevent this in the next tape-out?

ANSWER

```
// (a) Setup violation fixes (post-tape-out):
//   Option 1: Reduce operating frequency (most effective)
//     Setup slack = clock period - data path delay - t_setup
//     Increasing clock period directly adds slack
//   Option 2: Increase supply voltage (V_dd)
//     Higher voltage -> faster transistor switching -> shorter path delay
//   Option 3: Lower operating temperature
//     Lower temp -> less carrier scattering -> higher mobility -> faster
//   Option 4: Run at Fast PVT corner
```

```
//     Fast process + High voltage + Low temperature -> best case timing

// (b) Hold violation fixes (post-tape-out):
//   Option 1: Increase supply voltage
//     Faster clock-to-Q of launch FF -> data arrives later at capture FF
//     -> more hold margin (counterintuitive but works)
//   Option 2: Adjust clock phase / insert external clock delay
//     Delay capture clock -> widens hold window
//   Note: Reducing frequency does NOT help hold violations

// (c) Fundamental difference: FIXABILITY
//   Setup violation: FREQUENCY-DEPENDENT
//     -> Simply reduce clock frequency to add slack
//     -> Always fixable post-tape-out (at a performance cost)
//   Hold violation: FREQUENCY-INDEPENDENT
//     -> No matter how slow the clock, hold window stays the same
//     -> Cannot be fixed by frequency reduction
//     -> Post-tape-out options are very limited and unreliable
//     -> Hold violations are the most dangerous post-tape-out bugs

// (d) ECO (Engineering Change Order) for next tape-out:
//   Metal ECO  : add/change metal layers only (no diffusion changes)
//                can insert buffer/delay cells to fix hold
//   Full ECO   : full mask change, most expensive
//   Pre-tapeout: ensure signoff covers all PVT corners,
//                OCV/AOCV derating for on-chip variation,
//                explicit hold fixing step in PnR flow,
//                guard-band for hold at Fast corner
```

解释

Setup 和 Hold violation 在 tape-out 后的处理方式有本质区别。

Setup violation 本质上是时序预算不够：时钟周期太短，数据来不及。

降低频率就是增加时钟周期，直接给 setup 更多裕量，永远可以修复。

Hold violation 本质上是数据路径太快：数据在时钟边沿后太快就变了。

降低频率不能解决 hold violation，因为 hold 约束是：

"时钟边沿后，数据必须稳定至少 t_hold 时间"，这与时钟频率无关。

这就是为什么 hold violation 在 tape-out 后是"最危险的 bug"：

无法靠降频规避，只能靠升压、调相位等有限手段，可靠性很低。

ECO 流程：Metal ECO 只改金属层不改底层晶体管，成本较低，

可以在原有芯片上插入 buffer 来增加 hold 裕量，但只能在下次流片时用。

面试重点：一定要能清晰说明"hold 与频率无关"这个核心结论。

## Q44. Computer Architecture: Pipeline Advantages and Hazards

QUESTION

What is the advantage of using a pipeline in processor design?

Explain the three types of pipeline hazards, their causes,

and how each is resolved.

ANSWER

```
// Advantage of Pipelining:
// Without pipeline: each instruction takes N cycles (N stages).
//   Throughput = 1 instruction per N cycles.
// With pipeline: all N stages operate in parallel on different instr.
//   Throughput -> 1 instruction per cycle (ideal, after fill latency).
//   Latency of a single instruction stays N cycles, but throughput
//   increases by up to N times.

// Hazard Type 1: STRUCTURAL HAZARD
// Cause: Two instructions need the same hardware resource simultaneously.
//   Example: single memory port -- load and instruction fetch conflict.
// Solutions:
//   - Duplicate resources (separate I-cache and D-cache)
//   - Stall (pipeline bubble): delay one instruction by 1+ cycles

// Hazard Type 2: DATA HAZARD
// Cause: Instruction depends on result of a previous instruction
//        that has not yet completed (RAW, WAR, WAW).
//   RAW (Read After Write): most common -- instr B reads reg
//        that instr A is still writing.
//   Example: ADD R1, R2, R3  /  SUB R4, R1, R5  (R1 not ready)
// Solutions:
//   - Forwarding/Bypassing: route result directly from EX/MEM stage
//     back to EX input without waiting for WB
//   - Stall (NOP insertion): compiler or hardware inserts bubbles
//   - Out-of-order execution: reorder instructions to avoid dependency

// Hazard Type 3: CONTROL HAZARD (Branch Hazard)
// Cause: Pipeline fetches instructions before branch outcome is known.
//   Fetched instructions may need to be flushed if branch is taken.
// Solutions:
//   - Branch prediction (static/dynamic): speculate direction, flush
//     on mis-prediction
//   - Delayed branching: compiler fills branch delay slot with
//     useful instruction
//   - Stall until branch resolved (simple but costly)
//   - Flush pipeline on mis-prediction (modern out-of-order CPUs)
```

解释

流水线的核心价值：将一个指令的执行分拆成多个阶段，

使得不同指令的不同阶段可以并行执行，大幅提升吞吐量。

三种冲突的核心区别：

结构冲突：硬件资源不够（加硬件解决）。

数据冲突：指令间有数据依赖，最常考的是 RAW（先写后读），

解决方案是 Forwarding（数据旁路）——结果写入寄存器前直接转发给下一阶段。

控制冲突：分支方向未确定时已取指令，现代 CPU 用分支预测（Branch Prediction）应对，

预测错误时冲刷流水线（Pipeline Flush）。

## Verification

### Q45. Verification: Async FIFO Gray Code Pointer Checker

QUESTION

Given an asynchronous FIFO, how do you add checkers to verify

that the gray code pointer changes by only 1 bit at a time?

Implement both an SVA assertion and a SystemVerilog checker.

ANSWER

```
// SVA-based checker (inline assertion)
// Check write pointer (in write clock domain)
property p_wptr_gray_onehot;
  @(posedge wclk) disable iff (!wrst_n)
  $onehot($past(wptr_gray) ^ wptr_gray);
endproperty
assert property (p_wptr_gray_onehot)
  else $error("Write gray pointer changed by more than 1 bit!");

// Check read pointer (in read clock domain)
property p_rptr_gray_onehot;
  @(posedge rclk) disable iff (!rrst_n)
  $onehot0($past(rptr_gray) ^ rptr_gray);
  // $onehot0: allows no change (idle) OR exactly 1-bit change
endproperty
assert property (p_rptr_gray_onehot)
  else $error("Read gray pointer changed by more than 1 bit!");

// Standalone SystemVerilog checker class (for UVM environment)
class gray_ptr_checker;
  task check_gray(
    input logic [PTR_W-1:0] prev_gray,
    input logic [PTR_W-1:0] curr_gray,
    input string            domain
  );
    logic [PTR_W-1:0] diff = prev_gray ^ curr_gray;
    // Allow no-change (diff==0) or exactly 1-bit change ($onehot)
    if (diff != 0 && !$onehot(diff)) begin
      `uvm_error("GRAY_CHK",
        $sformatf("[%s] Gray pointer changed by %0d bits! prev=%0h curr=%0h",
                  domain, $countones(diff), prev_gray, curr_gray))
    end
  endtask
endclass

// Additional checks for a complete async FIFO verification plan:
// 1. Full/empty flag correctness (compare with binary pointer difference)
// 2. No overflow: write when full -> data must be rejected
// 3. No underflow: read when empty -> data must be invalid
// 4. Data integrity: write data matches read data in order
// 5. Pointer wrap-around: verify correct behavior at max depth
```

解释

异步 FIFO 的指针检查是验证中的重点：

写指针在写时钟域，读指针在读时钟域，两者分别用各自的时钟来检查。

$onehot：检查当前指针和上一个周期的异或结果是否恰好有一个 bit 为 1。

$onehot0：允许全 0（指针没有变化，即没有读写操作）或恰好 1 个 bit。

实际项目中这个 checker 通常和其他检查（overflow、underflow、数据完整性）一起组成完整的 FIFO 验证计划。

## Q46. Verification: How to Verify an Async FIFO

QUESTION

How do you verify the correctness of an asynchronous FIFO?

Describe the complete verification plan including:

test scenarios, checkers, assertions, and coverage.

ANSWER

```
// Complete Async FIFO Verification Plan

// 1. FUNCTIONAL TEST SCENARIOS
//   a. Basic write then read (single transaction)
//   b. Fill FIFO to full, check full flag asserts
//   c. Read from empty FIFO, check empty flag asserts
//   d. Simultaneous read and write at different clock rates
//   e. Write faster than read (f_wr >> f_rd): overflow stress test
//   f. Read faster than write (f_rd >> f_wr): underflow stress test
//   g. Pointer wrap-around: write/read enough to wrap PTR_W bits
//   h. Back-to-back transactions with no gap
//   i. Random clock frequencies and phase relationships
//   j. Reset during active operation (mid-write, mid-read)

// 2. ASSERTIONS (SVA)
//   - Gray pointer changes by exactly 1 bit (see Q51)
//   - full  flag: asserts when wptr_gray == rptr_gray_sync XOR MSB
//   - empty flag: asserts when rptr_gray == wptr_gray_sync
//   - No write when full (overflow protection)
//   - No read  when empty (underflow protection)
//   - Data read matches data written (in-order FIFO property)

// 3. SCOREBOARD / CHECKER
//   // Reference model: a simple SV queue mirrors FIFO behavior
//   class fifo_scoreboard;
//     logic [DATA_W-1:0] ref_q[$];
//
//     function void write(logic [DATA_W-1:0] data);
//       ref_q.push_back(data);
//     endfunction
//
//     function void read(logic [DATA_W-1:0] actual);
//       logic [DATA_W-1:0] expected;
//       if (ref_q.size() == 0)
//         `uvm_error("SCBD","Read from empty FIFO!")
//       else begin
//         expected = ref_q.pop_front();
//         if (expected !== actual)
//           `uvm_error("SCBD", $sformatf(
//             "Data mismatch: exp=%0h act=%0h", expected, actual))
//       end
//     endfunction
//   endclass

// 4. COVERAGE
//   covergroup fifo_cg;
//     cp_full:      coverpoint fifo_full;
//     cp_empty:     coverpoint fifo_empty;
//     cp_wr_rd_sim: coverpoint (wr_en && rd_en);   // simultaneous
//     cp_depth:     coverpoint occupancy {
//                     bins empty  = {0};
//                     bins full   = {DEPTH};
```

```
//                     bins others = {[1:DEPTH-1]};
//                   }
//     // Cross coverage: simultaneous read/write at various depths
//     cx_sim_depth: cross cp_wr_rd_sim, cp_depth;
//   endgroup

// 5. CLOCK DOMAIN SPECIFIC CHECKS
//   - Verify synchronized pointer arrives stable in destination domain
//   - Check metastability is handled (2-FF synchronizer present)
//   - Verify full/empty flags are glitch-free
//   - Boundary condition: exactly-full and exactly-empty
```

解释

异步 FIFO 验证的难点在于跨时钟域的指针传递和满/空信号的正确性。

完整的验证计划需要四个维度：

1. 功能场景：覆盖正常、边界、压力情况。

2. 断言：实时检查关键协议（Gray code、full/empty flag）。

3. Scoreboard：用一个 SV queue 作为参考模型，比较读出的数据是否正确。

4. 覆盖率：FIFO占用量、同时读写、满/空边界都需要覆盖。

面试重点：能条理清晰地说出上述四个维度，展示对完整验证流程的掌握。

==========

## Algorithms & Coding

### Q47. Find the k-th smallest element in an array

QUESTION

Given an unsorted integer array arr and an integer k, find the

k-th smallest element (k is 1-indexed).

Provide both a sort-based solution and QuickSelect.

Compare their time complexities.

Example: arr = [3, 1, 4, 1, 5, 9, 2, 6], k = 3 -> output: 3

ANSWER

```
// Solution 1: Sort-based  O(n log n)
function int kth_smallest_sort(int arr[], int k);
  arr.sort();
  return arr[k-1];
endfunction

// Solution 2: QuickSelect  average O(n), worst O(n^2)
function int partition(ref int arr[], int lo, int hi);
  int pivot = arr[hi], i = lo;
  for (int j = lo; j < hi; j++)
    if (arr[j] <= pivot) begin
      int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp; i++;
```

```
    end
  int tmp = arr[i]; arr[i] = arr[hi]; arr[hi] = tmp;
  return i;
endfunction

function int quickselect(ref int arr[], int lo, int hi, int k);
  if (lo == hi) return arr[lo];
  int p = partition(arr, lo, hi);
  if      (k == p) return arr[p];
  else if (k <  p) return quickselect(arr, lo, p-1, k);
  else             return quickselect(arr, p+1, hi, k);
endfunction
```

解释

排序法最简单，但 O(n log n) 做了很多不必要的工作，因为我们只需要第k小的。

QuickSelect 是 QuickSort 的变体，每次 partition 后只递归处理目标所在的一半，

平均时间复杂度 O(n)，是处理 top-k 问题的最优解。

注意：QuickSelect 最坏情况（每次选到最大/最小值做 pivot）是 $O(n^2)$，

可以用随机选 pivot 来规避。

面试中通常先写排序法，再说能用 QuickSelect 优化到 O(n)，展示思路递进。

## Q48. Reverse a stack using recursion only

QUESTION

Reverse a stack using ONLY recursion. No additional data structures

allowed. Hint: requires two recursive functions -- one to insert

an element at the bottom, one to reverse the entire stack.

Analyze time and space complexity.

Input (top->bottom): [1, 2, 3, 4, 5]

Output (top->bottom): [5, 4, 3, 2, 1]

ANSWER

```
// Step 1: insert element at the bottom of the stack
function void insert_at_bottom(ref int stk[$], int item);
  if (stk.size() == 0) begin
    stk.push_back(item);
  end else begin
    int top = stk.pop_back();
    insert_at_bottom(stk, item);
    stk.push_back(top);
  end
endfunction

// Step 2: reverse by popping top and inserting at bottom
function void reverse_stack(ref int stk[$]);
  if (stk.size() == 0) return;
  int top = stk.pop_back();
  reverse_stack(stk);
  insert_at_bottom(stk, top);
endfunction
```

```
// Time complexity:  O(n^2) -- reverse calls insert n times,
//                             each insert traverses up to n frames
// Space complexity: O(n)   -- call stack depth
```

解释

这道题的关键思路是把问题分解成两个递归函数：

insert_at_bottom：把一个元素插入栈底，方法是先把所有元素出栈，

插入目标元素，再把之前出栈的元素全部压回去。

reverse_stack：每次弹出栈顶元素，对剩余部分递归翻转，

再把弹出的元素插入到翻转后的栈底。

这两个函数组合起来实现了不用额外数据结构的栈翻转。

时间复杂度 $O(n^2)$ 是因为翻转 n 个元素，每次 insert_at_bottom 最多要

遍历 n 层调用栈。

## Q49. Maximum subarray sum (Kadane's Algorithm)

QUESTION

Given an integer array arr (may contain negatives), find the

maximum sum of a contiguous subarray.

Write the DP recurrence, and state time and space complexity.

Example: arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4] -> output: 6

(subarray: [4, -1, 2, 1])

ANSWER

```
// DP recurrence:
// dp[i] = max(arr[i], dp[i-1] + arr[i])
// answer = max(dp[0..n-1])

// Rolling-variable implementation (O(1) space):
function int max_subarray(int arr[]);
  int cur_max  = arr[0];
  int best_max = arr[0];
  for (int i = 1; i < arr.size(); i++) begin
    if (arr[i] > cur_max + arr[i])
      cur_max = arr[i];        // start fresh from arr[i]
    else
      cur_max = cur_max + arr[i];  // extend previous subarray
    if (cur_max > best_max)
      best_max = cur_max;
  end
  return best_max;
endfunction

// Time complexity:  O(n)
// Space complexity: O(1)
```

解释

Kadane 算法的核心思想：

对于每个位置 i，考虑两种选择：

1. 把 arr[i] 加入前面的子数组（cur_max + arr[i]）

2. 从 arr[i] 重新开始一个新的子数组（arr[i]）

选较大的那个，这就是 dp[i] = max(arr[i], dp[i-1] + arr[i])。

什么时候选择重新开始？当 dp[i-1] < 0 时，带着前面的累积反而会拖累，

不如从 arr[i] 重新开始。

这道题是动态规划的入门经典，O(n) 时间和 O(1) 空间，面试必须熟练写出。

## UVM

### Q50. OOP Design: Bicycle & Unicycle pricing system

QUESTION

You are building a bicycle rental pricing system.

Implement the following two classes:

Class Bicycle:

- Constructor accepts: size ("Adult" or "Kid"),

tire_type ("Tubular" / "Clincher" / "Tubeless"),

add_ons (list of strings, optional)

- Base prices : Adult = $60, Kid = $40

- Tire prices : Tubular = $10, Clincher = $20, Tubeless = $30

- Kids get 50% discount on tire price

- Each add-on : $15 for Adults, $10 for Kids

- Implement compute_cost() returning total as an integer

- Implement set_properties() to update fields after construction

Class Unicycle (inherits from Bicycle):

- Override base prices: Adult = $40, Kid = $20

- Unicycles cannot have "Tassels" or "Bell" as add-ons;

silently filter them out during construction and set_properties()

- All other pricing logic remains identical to Bicycle

Verify with these test cases:

Adult Unicycle: Clincher + ["Headlight", "Tassels"]

Expected: $40 + $20 + $15 = $75 (Tassels filtered out)

Kid Unicycle: Tubeless + ["Basket", "Bell"]

Expected: $20 + $15 + $10 = $45 (Bell filtered out, Tubeless * 0.5 = $15)

ANSWER

```
class Bicycle:
    """
```

```
    Base class for all bicycle types.

    Class-level price maps are defined here so subclasses can
    override them independently without touching any pricing logic.

    Pricing rules:
      - Base price depends on size (Adult / Kid).
      - Tire price applies in full for Adults, 50% discount for Kids.
      - Each add-on costs $15 for Adults and $10 for Kids.
    """

    BASE_PRICES = {"Adult": 60, "Kid": 40}
    TIRE_PRICES = {"Tubular": 10, "Clincher": 20, "Tubeless": 30}

    def __init__(self, size="Adult", tire_type="Clincher", add_ons=None):
        """
        Initialize a Bicycle instance.

        Args:
            size      (str): Rider size, either "Adult" or "Kid".
            tire_type (str): Tire type, one of "Tubular", "Clincher", "Tubeless".
            add_ons   (list[str], optional): List of accessory names.
                Defaults to an empty list if not provided.
        """
        self.size      = size.capitalize()
        self.tire_type = tire_type.capitalize()
        self.add_ons   = self._validate_addons(add_ons if add_ons else [])

    def _validate_addons(self, add_ons):
        """
        Validate and filter the add-ons list.

        Base Bicycle imposes no restrictions; all add-ons are accepted.
        Subclasses override this method to enforce their own rules
        (e.g., Unicycle disallows Tassels and Bell).

        Args:
            add_ons (list[str]): Requested add-on names.

        Returns:
            list[str]: Accepted add-on names after filtering.
        """
        return add_ons

    def set_properties(self, size=None, tire_type=None, add_ons=None):
        """
        Update one or more properties after construction.

        Only the arguments that are explicitly passed (not None) are
        updated.  add_ons go through _validate_addons so subclass
        restrictions are always enforced.

        Args:
            size      (str, optional): New size value.
            tire_type (str, optional): New tire type.
            add_ons   (list[str], optional): New add-ons list.
        """
        if size      is not None: self.size      = size.capitalize()
        if tire_type is not None: self.tire_type = tire_type.capitalize()
        if add_ons   is not None: self.add_ons   = self._validate_addons(add_ons)

    def compute_cost(self):
        """
```

```
        Calculate and return the total rental cost as an integer.

        Cost breakdown:
          1. Base price looked up from BASE_PRICES by size.
          2. Tire price looked up from TIRE_PRICES by tire_type;
             Kids receive a 50% discount.
          3. Add-on price: $15 per item for Adults, $10 for Kids.

        Returns:
            int: Total cost in dollars (truncated, not rounded).
        """
        total       = self.BASE_PRICES.get(self.size, 0)
        tire_cost   = self.TIRE_PRICES.get(self.tire_type, 0)
        total      += tire_cost * 0.5 if self.size == "Kid" else tire_cost
        add_on_rate = 10 if self.size == "Kid" else 15
        total      += len(self.add_ons) * add_on_rate
        return int(total)


class Unicycle(Bicycle):
    """
    Unicycle subclass with different base prices and add-on restrictions.

    Overrides:
      - BASE_PRICES: Adult $40, Kid $20 (cheaper than a Bicycle).
      - _validate_addons: silently removes "Tassels" and "Bell",
        which are physically incompatible with a unicycle.

    All other pricing logic (tire discount, add-on rate) is
    inherited unchanged from Bicycle.
    """

    BASE_PRICES = {"Adult": 40, "Kid": 20}

    def _validate_addons(self, add_ons):
        """
        Filter out add-ons that are incompatible with a unicycle.

        Tassels and Bell are silently ignored; no error is raised.
        Comparison is case-insensitive via capitalize().

        Args:
            add_ons (list[str]): Requested add-on names.

        Returns:
            list[str]: Accepted add-ons with restricted items removed.
        """
        restricted = {"Tassels", "Bell"}
        return [item for item in add_ons
                if item.capitalize() not in restricted]


# Test
uni1 = Unicycle(size="Adult", tire_type="Clincher",
                add_ons=["Headlight", "Tassels"])
print(f"Adult Unicycle: ${uni1.compute_cost()}")  # $75
# Base $40 + Clincher $20 + Headlight $15 = $75
# (Tassels silently removed by _validate_addons)

uni2 = Unicycle(size="Kid", tire_type="Tubeless",
                add_ons=["Basket", "Bell"])
print(f"Kid Unicycle:   ${uni2.compute_cost()}")  # $45
# Base $20 + Tubeless $30*0.5=$15 + Basket $10 = $45
```

```
# (Bell silently removed by _validate_addons)
```

解释

这道题考查 Python OOP 的三个核心机制：

1. 类变量的继承覆盖：

BASE_PRICES 是类变量（class-level），定义在类体而非 __init__ 里。

Unicycle 在子类中重新定义 BASE_PRICES，覆盖父类版本。

实例访问 self.BASE_PRICES 时，Python 先找实例 __dict__，

找不到再找类 __dict__，所以 Unicycle 实例会找到子类的版本。

2. 模板方法模式（Template Method Pattern）：

_validate_addons 是一个钉子方法（hook method）。

父类定义了调用它的时机（__init__ 和 set_properties），

子类只需覆盖这一个方法，就改变了所有涉及 add_ons 的行为。

compute_cost 和 set_properties 完全不需要改，体现了开闭原则。

3. 多态（Polymorphism）：

uni1 和 uni2 都调用 compute_cost()，

但 add_ons 的过滤逻辑自动走 Unicycle 的版本。

调用方不需要知道对象是 Bicycle 还是 Unicycle，接口完全一致。

面试重点：

- 能解释为什么选择覆盖 _validate_addons 而不是覆盖 compute_cost
- 理解 Python MRO 如何找到正确的 BASE_PRICES
- capitalize() 确保输入大小写不影响字典查找

## Brain Teasers

### Q51. Brain teaser: measure 4L using 5L and 3L buckets

QUESTION

You have a 5L bucket and a 3L bucket with unlimited water supply.

How do you measure exactly 4L of water?

Write out the complete steps.

ANSWER

```
Step 1: Fill the 5L bucket.          (5L:5, 3L:0)
Step 2: Pour from 5L into 3L bucket until 3L is full.
                                      (5L:2, 3L:3)
Step 3: Empty the 3L bucket.         (5L:2, 3L:0)
Step 4: Pour the 2L from 5L into 3L. (5L:0, 3L:2)
Step 5: Fill the 5L bucket again.    (5L:5, 3L:2)
```

```
Step 6: Pour from 5L into 3L until 3L is full (only 1L fits).
                                    (5L:4, 3L:3)
Result: 5L bucket contains exactly 4L.
```

解释

这道题考查逻辑推理和状态搜索能力。

核心思路：通过多次倒水和清空，利用两个桶的容量差（5-3=2）来构造目标量。

另一种解法：

步骤1：装满3L桶，倒入5L桶（5L:3, 3L:0）

步骤2：再装满3L桶，倒入5L桶直到满（5L:5, 3L:1）

步骤3：清空5L桶，把3L桶剩余的1L倒入5L桶（5L:1, 3L:0）

步骤4：装满3L桶，倒入5L桶（5L:4, 3L:0）

两种方法都能得到4L，面试时说明一种即可，展示思路清晰即可。

## Q52. Brain Teaser: The Three Light Switches

QUESTION

There are three light switches in your basement. All of them are currently in the off position. Each switch controls its own lamp on the floor above. You need to find out which switch controls which lamp. You are not allowed to break walls or inspect wiring.

Rules:

- You must start in the basement where the switches are located
- You may turn switches on or off any number of times
- You may only go upstairs ONE time to inspect the lamps
- At the time you go upstairs you must make your final decision

How can you determine which switch controls each lamp with only one trip upstairs?

Hint: you may need to think outside the box, but the answer is still logical.

ANSWER

```
Step 1: Turn on Switch A and leave it on for ~10 minutes.
Step 2: Turn Switch A OFF.
Step 3: Turn Switch B ON.
Step 4: Leave Switch C OFF. Go upstairs immediately.

Now inspect the three lamps:
- The lamp that is ON         -> controlled by Switch B
                                 (currently on)
- The lamp that is OFF + WARM -> controlled by Switch A
                                 (was on, now off, still warm)
- The lamp that is OFF + COLD -> controlled by Switch C
                                 (never turned on)
```

解释

这道题的关键在于打破思维定势：

大多数人只想到用开/关状态（2种）来区分三个射灯，

但 2 种状态无法区分 3 个射灯。

突破口在于利用灯泡的热量作为第三个维度：

亮着（on）、息灯且热（刚关过）、息灯且冷（从未开过）

这样三种状态恰好区分三个开关。

这也是面试中考察候选人是否能利用多个维度的信息解决问题的经典考题，

属于 lateral thinking（横向思维）类型。

## Additional Questions

==========

END OF DOCUMENT

==========